

Simulación de Rutas Aéreas con Grafos Dirigidos

Red Nacional de Vuelos de México

Proyecto de la materia de Estructuras de Datos / Programación

Nombre del alumno: [TU NOMBRE]

Grupo / Turno: [TU GRUPO]

Fecha: [FECHA]

1. Introducción

Un **grafo** es una estructura matemática formada por un conjunto de **nodos** (también llamados vértices) y un conjunto de **aristas** (también llamadas conexiones o arcos) que unen pares de nodos. Los grafos se utilizan para representar relaciones entre objetos: cada nodo representa un elemento y cada arista representa una relación o conexión entre dos de ellos. Por ejemplo, en un mapa de carreteras los nodos son las ciudades y las aristas son las carreteras que las conectan.

Un **grafo dirigido** (también llamado digrafo) es un grafo en el que cada arista tiene una **dirección**: va desde un nodo de origen hasta un nodo de destino. Esto significa que la conexión no es simétrica; es decir, si existe una arista de A hacia B, no necesariamente existe una arista de B hacia A. Los grafos dirigidos son ideales para modelar situaciones en las que el sentido importa, como el tráfico en calles de un solo sentido, las redes sociales (donde se puede seguir a alguien sin que esa persona te siga) o los vuelos de una aerolínea.

Algunos **ejemplos de uso de los grafos en la vida real** son:

- **Mapas y navegadores (GPS):** las calles y carreteras forman un grafo donde los nodos son las intersecciones y las aristas son las vías; los algoritmos de ruta más corta (como el de Dijkstra) se usan para calcular el mejor camino.
- **Redes sociales:** los usuarios son nodos y las amistades o seguimientos son aristas (en el caso de los seguimientos, aristas dirigidas).
- **Internet y telecomunicaciones:** las computadoras o servidores son nodos y los cables o enlaces inalámbricos son las aristas por donde viajan los datos.
- **Rutas aéreas:** las ciudades son nodos y los vuelos entre ellas son las aristas dirigidas; es exactamente el problema que se simula en este proyecto.
- **Redes eléctricas y de agua potable:** las centrales, las subestaciones y los hogares se conectan mediante redes que se representan como grafos.
- **Organigramas y procesos:** las jerarquías de una empresa o los diagramas de flujo de un algoritmo también son grafos dirigidos.

2. Justificación

Los **grafos son la herramienta ideal para representar redes de transporte y rutas aéreas** por varias razones:

- **Modelado natural:** una red de vuelos se compone de ciudades (nodos) y conexiones entre ellas (aristas). La correspondencia es directa: cada ciudad del país es un nodo y cada vuelo es una arista.
- **Direccionalidad:** un vuelo de la Ciudad de México a Cancún no implica que exista el vuelo de regreso en el mismo horario o por la misma aerolínea. El uso de un **grafo dirigido** permite representar exactamente qué rutas existen en cada sentido, igual que en la realidad.
- **Cálculo de rutas óptimas:** con algoritmos de teoría de grafos, como el de Dijkstra, se puede calcular la ruta más corta (por kilómetros o por número de escalas) entre dos ciudades, que es el problema central de cualquier aerolínea o agencia de viajes.
- **Análisis de la red:** los grafos permiten identificar ciudades "hub" (las más

conectadas), saber si la red es conexas o si alguna ciudad quedaría incomunicada si se cancela una ruta.

- **Generalización:** los mismos modelos se usan en logística, mensajería, reparto de paquetería y transporte terrestre, por lo que lo aprendido se aplica a muchos problemas reales.

3. Desarrollo del Proyecto

3.1 Cómo funciona el programa

El programa se ejecuta desde la terminal con `python3 main.py`. Al iniciar, construye automáticamente una red inicial con 13 ciudades principales de México (Ciudad de México, Guadalajara, Monterrey, Tijuana, Cancún, Mérida, Veracruz, Oaxaca, Puebla, León, Puerto Vallarta, La Paz y San Luis Potosí) y 25 rutas de ejemplo. A continuación muestra un menú con 9 opciones que permite al usuario administrar la red completa. Todas las ciudades precargadas son editables: se pueden eliminar o añadir rutas, igual que cualquier ciudad agregada después.

El sistema incluye un **catálogo de 65 ciudades de México** con sus coordenadas geográficas reales (latitud y longitud). Al agregar una ciudad nueva, el usuario la busca por nombre (sin necesidad de escribir acentos) y el programa la coloca automáticamente en su posición real sobre el mapa.

3.2 Cómo está estructurado el grafo

El grafo se construye con la librería **NetworkX** usando la clase `nx.DiGraph()`, es decir, un **grafo dirigido**:

- Cada **nodo** es una ciudad y almacena como atributo su posición geográfica `pos = (latitud, longitud)`.
- Cada **arista dirigida** es un vuelo y almacena como atributo su distancia en kilómetros (km), calculada con la **fórmula de Haversine** a partir de las coordenadas de las dos ciudades.
- Con la distancia y una velocidad de crucero promedio de 850 km/h se estima la **duración del vuelo** (más 45 minutos de despegue y aterrizaje).
- El territorio mexicano se dibuja con **Matplotlib** usando los polígonos (anillos de coordenadas) que forman la frontera del país; las ciudades se colocan en sus coordenadas reales, simulando un mapa tipo "Google Maps".

3.3 Cómo funciona el menú

Op
ció

Descripción

n

1.
Ag
reg
ar
Ci
ud
ad

Muestra el catálogo de 65 ciudades de México con búsqueda por nombre; el usuario elige una y se agrega al grafo en sus coordenadas reales.

2.
Ag
reg
ar
Ru
ta
de
Vu
elo

Pide ciudad de origen y destino (acepta nombres con o sin acentos), valida que ambas existan y crea la arista dirigida con su distancia. Pregunta si se desea crear también el vuelo de regreso (ida y vuelta).

3.
Eli
mi
nar

Elimina una ciudad y todas sus rutas asociadas (incluye las precargadas).

Ci
ud
ad

4.
Eli
mi
nar
Ru
ta

Elimina una conexión específica entre dos ciudades.

5.
Mo
str
ar
Ma
pa
de
Ru
tas

Dibuja la red completa sobre el mapa de México: fronteras del país, ciudades en sus coordenadas, flechas dirigidas y distancia en km sobre cada ruta.

6.
Ru
ta

Calcula la mejor ruta entre dos ciudades minimizando la distancia (kilómetros) o el número de escalas, y la resalta en rojo sobre el mapa.

Má
s
Co
rta
(Di
jks
tra
)

7.
Dis
tan
cia
y
Du
rac
ión

Muestra kilómetros y tiempo estimado de un vuelo directo o del mejor recorrido con escalas.

8.
Lis
tar
Ci
ud
ad
es
y
Ru
tas

Imprime en consola todas las ciudades con sus coordenadas y todas las rutas con su distancia.

9.
Sal
ir

Termina el programa.

Si el usuario selecciona una opción inválida (no numérica o fuera de rango), el programa muestra un mensaje de error y vuelve a pedir la opción.

3.4 Qué hace cada función del programa

- `normalizar_nombre()`: limpia el nombre de una ciudad (espacios, mayúsculas correctas sin capitalizar preposiciones como "de" o "del").
- `plegar_acentos()`: convierte el texto a minúsculas sin acentos para que "Ciudad Juárez" se encuentre escribiendo "juarez".
- `distancia_haversine()`: calcula la distancia en km entre dos puntos geográficos con la fórmula de Haversine.
- `formato_duracion()` y `formato_km()`: dan formato legible a horas y kilómetros.
- `RedVueLos.__init__()`: crea el grafo dirigido y precarga las ciudades y rutas iniciales.
- `_agregar_nodo()` / `_agregar_arista()`: internas; agregan nodos con posición y aristas con distancia.
- `_buscar_ciudad()`: encuentra el nombre canónico de una ciudad tolerando mayúsculas y acentos.
- `agregar_ciudad()`: agrega una ciudad del catálogo al grafo validando que no

exista.

- `agregar_ruta()`: crea la arista dirigida con su distancia en km y valida ciudades y duplicados.
- `eliminar_ciudad()`: elimina el nodo y todas sus aristas.
- `eliminar_ruta()`: elimina una arista específica.
- `ruta_mas_corta()`: aplica el algoritmo de **Dijkstra** (por km) o BFS (por escalas) entre dos ciudades.
- `distancia_y_duracion()`: muestra km y tiempo del vuelo directo o del mejor recorrido con escalas.
- `listar_red()`: imprime el estado completo de la red.
- `mostrar_grafo()`: dibuja el mapa de México con la red, las distancias y (opcionalmente) una ruta resaltada en rojo.
- `mostrar_menu()`: imprime el menú de opciones.
- `seleccionar_ciudad_del_catalogo()`: despliega el catálogo con búsqueda y devuelve la ciudad elegida.
- `principal()`: bucle principal que lee la opción del usuario, valida errores y ejecuta la acción correspondiente.

4. Código del Programa

A continuación se incluye el código completo del programa en Python. Las librerías necesarias son `networkx` y `matplotlib` (instalación: `pip install networkx matplotlib`).

```
"""
```

```
=====
```

```
Sistema de Administración de Rutas Aéreas de México
```

```
Simulación de rutas aéreas nacionales mediante grafos dirigidos.
```


Cada ciudad es un nodo (con sus coordenadas reales de latitud/longitud) y

cada vuelo es una arista dirigida (con su distancia en kilómetros).

La red se dibuja sobre el contorno geográfico de la República Mexicana,

simulando un mapa estilo Google Maps.

Librerías utilizadas: NetworkX (grafo) y Matplotlib (visualización).

=====

"""

```
import math
```

```
import matplotlib.pyplot as plt
```

```
import networkx as nx
```

```
# -----
```

```
# 1. DATOS GEOGRÁFICOS
```

```
# -----
```

```
# Contorno de la República Mexicana (polígonos de los 32 estados, simplificados).
```

```
# Cada anillo es una lista de puntos (longitud, latitud).
```

```
FRONTERA_MEXICO = [ # lista de polígonos (anillos) del contorno del país
```

```
[
```

(-117.1361,32.5776), (-114.7131,32.7629), (-114.8031,32.6626),
(-114.8113,32.5225), (-114.9327,32.5214), (-115.0531,32.2800),

(-114.9738,32.2114), (-114.9638,31.9467), (-114.8020,31.8429),
(-114.7784,31.6581), (-114.8541,31.5210), (-114.8874,31.1632),

(-114.8246,31.0030), (-114.7032,30.9137), (-114.6223,30.4884),
(-114.6601,30.1851), (-114.5266,29.9597), (-114.4002,29.8784),

(-114.3727,29.7625), (-114.1913,29.7286), (-113.6093,29.2589),
(-113.4957,28.8610), (-113.3525,28.9083), (-113.3208,28.7704),

(-113.2156,28.8072), (-113.1594,28.7580), (-113.0654,28.4661),
(-112.8027,28.3996), (-112.8300,28.2377), (-112.7412,28.1499),

(-112.7192,27.9499), (-114.1880,27.9499), (-114.0425,28.1616),
(-114.1065,28.2119), (-114.0497,28.4753), (-114.9654,29.3525),

(-115.1913,29.4063), (-115.7129,29.7402), (-115.7150,29.8882),
(-115.8280,29.9383), (-115.8271,30.2964), (-115.9342,30.3986),

(-116.0197,30.3570), (-116.0672,30.4698), (-116.0776,30.8015),
(-116.3559,30.9695), (-116.3342,31.1636), (-116.3904,31.2634),

(-116.7239,31.5752), (-116.6819,31.6877), (-116.7785,31.7681),
(-116.6880,31.7466), (-116.6438,31.8671), (-116.9169,32.0489),

(-116.9603,32.2613), (-117.0776,32.3192), (-117.1597,32.5033),
(-117.1361,32.5776),

],

[

(-113.5162,29.5548), (-113.3845,29.4606), (-113.3633,29.3071),
(-113.1556,29.2764), (-113.1101,28.9791), (-113.5919,29.4094),

(-113.5755,29.5414), (-113.5162,29.5548),

],

[

(-114.9686,32.5554), (-111.0849,31.3726), (-108.7194,31.3724),
(-108.8731,31.2456), (-108.7695,31.2302), (-108.8109,31.1716),

(-108.6458,31.0395), (-108.7677,31.0383), (-108.7668,30.9355),
(-108.9064,30.9391), (-108.9156,30.7650), (-108.8326,30.7165),

(-108.8423,30.6504), (-108.6397,30.5816), (-108.6264,30.4909),
(-108.5643,30.4878), (-108.5829,30.3605), (-108.5014,30.3025),

(-108.5810,30.3176), (-108.5968,29.8396), (-108.4940,29.7598),
(-108.6206,29.7238), (-108.5860,29.5044), (-108.6805,29.4686),

(-108.6315,29.3069), (-108.7255,29.2523), (-108.6609,29.1237),
(-108.6723,28.8807), (-108.5438,28.7283), (-108.4376,28.3503),

(-108.7559,28.2321), (-109.0384,28.2652), (-109.0427,28.1249),
(-108.8349,27.8094), (-108.8618,27.7131), (-108.7485,27.6984),

(-108.6138,27.5011), (-108.5512,27.2778), (-108.6242,27.1773),
(-108.5627,26.9910), (-108.3811,26.9291), (-108.6408,26.5321),

(-108.8390,26.4955), (-108.8474,26.3477), (-108.9749,26.3438),
(-109.0961,26.2363), (-109.2336,26.2595), (-109.2414,26.4450),

(-109.3794,26.5999), (-109.7549,26.6567), (-109.9549,27.0086),
(-110.5970,27.2710), (-110.6168,27.7788), (-110.5254,27.8212),

(-110.8586,27.8839), (-110.8038,27.8919), (-110.8338,27.9268),
(-110.9137,27.8689), (-110.8613,27.8713), (-110.8961,27.8099),

(-110.9920,27.9310), (-111.1154,27.9088), (-111.2685,28.0286),
(-111.4828,28.3496), (-111.7349,28.4415), (-111.7983,28.5926),

(-111.9775,28.7458), (-111.9274,28.7761), (-112.2041,28.9617),
(-112.2662,29.3072), (-112.4462,29.3446), (-112.4168,29.4900),

(-112.7072,29.9029), (-112.7906,29.9271), (-112.8032,30.2159),
(-112.9100,30.2869), (-112.9103,30.4356), (-113.1343,30.7054),

(-113.1519,31.2266), (-113.6855,31.3679), (-113.7015,31.5235),
(-113.9179,31.6407), (-114.0474,31.6774), (-114.0237,31.6070),

(-114.0948,31.5354), (-114.2410,31.5422), (-114.6628,31.8006),
(-114.9973,31.9185), (-115.0531,31.9702), (-115.0632,32.2349),

(-115.1424,32.3034), (-114.9686,32.5554),

],

[

(-112.2905,29.2309), (-112.1860,28.9766), (-112.2515,28.7633),
(-112.3092,28.7419), (-112.5889,28.8743), (-112.4921,28.9481),

(-112.4636,29.1922), (-112.2905,29.2309),

],

[

(-112.7787,28.0231), (-112.7921,27.9074), (-112.6948,27.7642),
(-112.3419,27.5562), (-112.2220,27.2361), (-111.9577,27.1265),

(-112.0208,27.0280), (-111.9876,26.9148), (-111.8028,26.9149),
(-111.5581,26.7189), (-111.5621,26.5782), (-111.4362,26.5364),

(-111.4637,26.4299), (-111.3156,26.1103), (-111.3511,25.9671),
(-111.2976,25.7771), (-111.1092,25.5269), (-111.0024,25.5134),

(-110.8910,25.1369), (-110.6642,24.8699), (-110.6355,24.7905),
(-110.7182,24.5320), (-110.5896,24.2298), (-110.2964,24.1468),

(-110.3061,24.3037), (-110.2041,24.3192), (-109.9736,24.1308),
(-109.9493,24.0128), (-109.7886,24.0300), (-109.7995,23.8996),

(-109.6586,23.7631), (-109.6563,23.6241), (-109.4230,23.5139),
(-109.3667,23.3429), (-109.4098,23.1538), (-109.8072,22.8478),

(-109.9284,22.8178), (-110.0276,22.8967), (-110.1368,23.2716),
(-110.3137,23.5302), (-110.5767,23.6610), (-111.0078,24.0636),

(-111.2662,24.2166), (-111.5744,24.3431), (-111.7108,24.2775),
(-112.1094,24.5381), (-112.1836,24.7488), (-112.3139,24.7757),

(-112.1326,25.2757), (-112.1015,25.6862), (-112.2006,25.9942),
(-112.4082,26.2471), (-112.6988,26.3298), (-113.3706,26.8106),

(-113.4984,26.8327), (-113.6446,26.7361), (-113.8951,26.9980),
(-114.0478,26.9987), (-114.2302,27.1695), (-114.4849,27.2061),

(-114.5690,27.4537), (-115.0716,27.7602), (-115.1492,27.8926),
(-114.5545,27.8156), (-114.3460,27.9096), (-114.2542,28.0482),

(-112.7787,28.0231),

],

[

(-101.8299,25.0611), (-101.7530,24.9161), (-101.5774,24.8754),
(-101.5988,24.7972), (-101.4350,24.7746), (-101.3131,24.8527),

(-101.1834,24.8137), (-101.0462,24.6261), (-100.7600,24.5904),
(-100.7013,24.4927), (-100.9083,24.3099), (-100.9709,24.3656),

(-100.9512,24.4642), (-101.0308,24.2342), (-101.1852,24.1860),
(-101.1604,23.9400), (-101.7547,23.4597), (-102.1091,23.3640),

(-102.2270,23.4589), (-102.2860,23.3300), (-102.2203,23.3245),
(-102.2792,23.2394), (-102.2025,23.1370), (-102.2493,23.0499),

(-102.1801,22.8653), (-101.9226,22.6417), (-101.8344,22.6522),
(-101.8545,22.5574), (-101.7793,22.4630), (-101.6372,22.4985),

(-101.4649,22.7408), (-101.3203,22.7091), (-101.3517,22.6724),
(-101.2555,22.5011), (-101.4079,22.3314), (-101.2948,22.1972),

(-101.3545,22.0042), (-101.2981,21.9800), (-101.5118,21.7897),
(-101.5453,21.8985), (-101.6278,21.8889), (-101.7256,21.9954),

(-102.0105,22.1154), (-102.0221,22.2880), (-102.1404,22.2793),
(-102.1369,22.3372), (-102.3021,22.4491), (-102.3119,22.3794),

(-102.4637,22.2985), (-102.6662,22.2808), (-102.6888,22.0918),
(-102.8757,21.8329), (-102.7443,21.6975), (-102.8055,21.6478),

(-102.7641,21.5711), (-102.6274,21.5077), (-102.6395,21.3376),
(-102.7278,21.3582), (-102.9130,21.1724), (-103.0709,21.2240),

(-103.1046,21.0941), (-103.0617,21.0644), (-103.1340,21.0177),
(-103.2179,21.0592), (-103.3643,21.0092), (-103.4412,21.1127),

(-103.5844,21.0821), (-103.6848,21.1834), (-103.6508,21.2510),
(-103.7393,21.2772), (-103.5181,21.3475), (-103.5536,21.4167),

(-103.6113,21.3618), (-103.7196,21.4387), (-103.7077,21.5265),
(-103.5602,21.5625), (-103.5227,21.7176), (-103.5882,21.7826),

(-103.4840,21.7929), (-103.2882,21.9725), (-103.1178,21.9976),
(-103.1599,22.0725), (-103.0827,22.1111), (-103.1123,22.1783),

(-103.0607,22.2353), (-103.2291,22.2692), (-103.1760,22.3758),
(-103.2950,22.4056), (-103.3781,22.3411), (-103.3051,22.2414),

(-103.4212,22.2028), (-103.4524,22.1031), (-103.7016,22.0945),
(-103.6109,22.2616), (-103.6590,22.3308), (-103.5745,22.4201),

(-103.6189,22.5510), (-103.7452,22.5687), (-103.8843,22.1533),
(-103.9487,22.3218), (-103.7711,22.7090), (-103.9316,22.7482),

(-104.0992,22.6940), (-103.9521,22.4687), (-104.0395,22.3552),
(-104.1234,22.3481), (-104.1076,22.5273), (-104.2432,22.4769),

(-104.1948,22.3782), (-104.2493,22.4126), (-104.3220,22.3339),
(-104.3847,22.3924), (-104.2907,22.5810), (-104.2958,22.7343),

(-104.1062,22.7650), (-104.1997,23.1341), (-104.0914,23.4758),
(-103.8991,23.6446), (-103.8135,23.6465), (-103.8049,23.6932),

(-103.8722,23.6963), (-103.8388,23.9873), (-103.8939,24.0302),
(-103.6777,24.1754), (-103.6385,24.1429), (-103.5838,24.3152),

(-103.4614,24.3623), (-103.4318,24.4622), (-103.1046,24.4327),
(-103.0481,24.4989), (-102.8938,24.4934), (-102.8325,24.4155),

(-102.6980,24.4045), (-102.4740,24.4597), (-102.5616,24.9433),
(-102.6706,24.9847), (-102.6255,25.0516), (-102.6672,25.1110),

(-102.4398,25.0426), (-102.4567,25.1613), (-102.2855,25.1708),
(-101.8299,25.0611),

],

[

(-105.9871,26.8223), (-105.7917,26.6952), (-105.7617,26.7492),
(-105.6888,26.6510), (-105.6134,26.6482), (-105.6323,26.7182),

(-105.4813,26.5399), (-105.3764,26.5660), (-105.2704,26.4923),
(-105.1040,26.5528), (-105.0582,26.4487), (-104.8059,26.5397),

(-104.5663,26.3683), (-104.4239,26.6376), (-104.2727,26.7609),
(-104.2589,26.8801), (-104.2128,26.8137), (-103.6318,26.7549),

(-103.3421,26.6354), (-103.2264,26.3114), (-103.3179,26.1662),
(-103.3066,25.7228), (-103.4647,25.5495), (-103.3040,25.3797),

(-103.4317,25.3848), (-103.4813,25.2857), (-103.0513,24.8186),
(-102.8139,24.7356), (-102.6235,25.0690), (-102.5819,25.0095),

(-102.6269,24.9426), (-102.5180,24.9013), (-102.4303,24.4176),
(-102.6543,24.3625), (-102.7888,24.3734), (-102.8501,24.4514),

(-103.0044,24.4568), (-103.0609,24.3907), (-103.3881,24.4201),

(-103.4177,24.3203), (-103.5401,24.2731), (-103.5948,24.1008),

(-103.6340,24.1333), (-103.8502,23.9882), (-103.7951,23.9452),
(-103.8285,23.6542), (-103.7612,23.6511), (-103.7698,23.6045),

(-103.8554,23.6026), (-104.0477,23.4338), (-104.1560,23.0920),
(-104.0625,22.7230), (-104.2521,22.6923), (-104.2854,22.4119),

(-104.4738,22.2974), (-104.7851,22.6019), (-104.8560,22.6023),
(-104.9881,22.4497), (-105.0702,22.6330), (-104.8910,22.6932),

(-104.9886,22.7938), (-105.0036,22.9375), (-105.3304,22.9012),
(-105.4246,23.0802), (-105.6573,23.1056), (-105.6433,23.2013),

(-105.7851,23.5286), (-105.8283,23.5843), (-105.9275,23.5610),
(-105.9731,23.8878), (-105.8751,23.8783), (-105.9647,23.9280),

(-105.9308,23.9785), (-106.0191,24.0253), (-106.0978,24.2578),
(-106.2946,24.3633), (-106.5478,24.2737), (-106.6340,24.4039),

(-106.5916,24.4632), (-106.7405,24.6596), (-107.1729,25.0552),
(-107.1639,25.2490), (-107.2524,25.3392), (-107.1825,25.5004),

(-107.2178,25.5687), (-107.0346,25.6765), (-106.9321,25.6582),
(-106.9116,25.5843), (-106.7425,25.6155), (-106.6863,25.5710),

(-106.4933,25.7446), (-106.5875,25.7066), (-106.5917,25.9320),
(-106.4853,25.9930), (-106.4618,26.2291), (-106.3132,26.3628),

(-106.2673,26.4972), (-106.3061,26.5637), (-106.2325,26.7126),
(-106.1537,26.7373), (-106.2016,26.8088), (-106.1231,26.7955),

(-106.0687,26.8828), (-105.9871,26.8223),

],

[

(-107.6735,31.8553), (-106.5229,31.8554), (-106.3727,31.8029),
(-106.1961,31.5329), (-105.9364,31.4273), (-105.3730,30.9062),

(-105.1837,30.8580), (-104.8846,30.6515), (-104.8201,30.4342),
(-104.6641,30.2748), (-104.6404,29.9637), (-104.4623,29.6612),

(-104.1609,29.5107), (-103.9824,29.3422), (-103.7228,29.2868),
(-103.6564,29.2007), (-103.2368,29.0402), (-103.4486,28.6742),

(-103.5321,28.6482), (-103.8690,27.9667), (-103.8345,27.9082),
(-103.8979,27.9458), (-103.7955,27.5364), (-103.8182,27.2938),

(-103.6794,26.9989), (-103.7925,26.8700), (-103.6369,26.6957),
(-104.1762,26.7486), (-104.2223,26.8151), (-104.2361,26.6959),

(-104.3873,26.5726), (-104.5297,26.3032), (-104.7693,26.4746),
(-105.0216,26.3836), (-105.0674,26.4878), (-105.2338,26.4273),

(-105.3398,26.5009), (-105.4447,26.4748), (-105.5957,26.6531),
(-105.5768,26.5831), (-105.6522,26.5860), (-105.7251,26.6841),

(-105.7551,26.6301), (-106.0321,26.8177), (-106.0864,26.7305),
(-106.1650,26.7437), (-106.1171,26.6722), (-106.1959,26.6476),

(-106.2695,26.4987), (-106.2307,26.4321), (-106.2766,26.2977),
(-106.4252,26.1640), (-106.4487,25.9280), (-106.5420,25.9074),

(-106.5509,25.6415), (-106.4567,25.6796), (-106.6497,25.5060),
(-106.7059,25.5504), (-106.8750,25.5192), (-106.8955,25.5931),

(-106.9980,25.6114), (-106.9684,25.6478), (-107.0478,25.6476),
(-107.3058,25.8792), (-107.3573,26.0677), (-107.5342,26.0623),

(-107.5975,26.1300), (-107.8001,26.0956), (-107.8749,26.1564),
(-108.0368,26.4038), (-107.9840,26.4934), (-108.1070,26.6108),

(-108.0296,26.7322), (-108.1698,26.8820), (-108.2586,26.8798),
(-108.2228,27.0093), (-108.5134,27.0163), (-108.5363,26.9630),

(-108.6381,27.0136), (-108.6996,27.1999), (-108.6267,27.3004),
(-108.6892,27.5237), (-108.8240,27.7210), (-108.9372,27.7356),

(-108.9103,27.8320), (-109.1181,28.1475), (-109.1139,28.2878),
(-108.8313,28.2546), (-108.5131,28.3729), (-108.6193,28.7509),

(-108.7478,28.9033), (-108.7363,29.1462), (-108.8009,29.2749),
(-108.7069,29.3295), (-108.7559,29.4912), (-108.6615,29.5270),

(-108.6961,29.7464), (-108.5695,29.7823), (-108.6722,29.8622),
(-108.6564,30.3401), (-108.5768,30.3251), (-108.6583,30.3831),

(-108.6397,30.5104), (-108.7019,30.5135), (-108.7151,30.6042),
(-108.9178,30.6730), (-108.9081,30.7391), (-108.9911,30.7876),

(-108.9818,30.9617), (-108.8422,30.9581), (-108.8431,31.0609),
(-108.7213,31.0620), (-108.8863,31.1942), (-108.8449,31.2528),

(-108.9482,31.2736), (-108.7948,31.3950), (-108.2364,31.3960),
(-108.2363,31.8554), (-107.6735,31.8553),

],

[

(-104.3398,19.0278), (-104.3085,19.0702), (-104.3627,19.1157),
(-104.4492,19.0822), (-104.7014,19.1850), (-104.6003,19.1504),

(-104.5395,19.2622), (-104.1241,19.3227), (-104.0616,19.5049),

(-103.8143,19.4142), (-103.6093,19.5200), (-103.4748,19.3419),

(-103.4772,18.9586), (-103.6215,18.8803), (-103.6111,18.7947),
(-103.7311,18.6751), (-104.3398,19.0278),

],

[

(-105.3930,23.1050), (-105.3318,22.9618), (-105.0050,22.9982),
(-104.9899,22.8544), (-104.8923,22.7538), (-105.0715,22.6936),

(-104.9894,22.5104), (-104.8573,22.6629), (-104.7864,22.6626),
(-104.4984,22.3537), (-104.3607,22.4317), (-104.2807,22.3741),

(-104.3590,21.9759), (-104.1789,21.9868), (-104.1467,21.8077),
(-103.9037,21.7689), (-103.8994,21.5753), (-103.6972,21.4161),

(-103.7975,21.2351), (-104.2055,21.1618), (-104.1925,20.9167),
(-104.3652,20.7614), (-104.2703,20.7608), (-104.2405,20.5771),

(-104.5498,20.9043), (-104.7799,21.0055), (-104.9683,20.8893),
(-105.1024,20.8995), (-105.2836,20.6471), (-105.3568,20.7367),

(-105.5506,20.7389), (-105.2453,21.0391), (-105.2549,21.3359),
(-105.1936,21.4510), (-105.4553,21.6189), (-105.6427,21.9585),

(-105.6511,22.1872), (-105.7773,22.5429), (-105.6923,22.4791),
(-105.7158,22.5768), (-105.6568,22.6074), (-105.4523,22.5364),

(-105.5370,22.7919), (-105.4858,22.8701), (-105.5130,22.9794),
(-105.3930,23.1050),

],

[

(-103.4368,18.2645), (-103.7510,18.5812), (-103.7777,18.6803),
(-103.6574,18.7897), (-103.6678,18.8753), (-103.6096,18.8611),

(-103.5075,18.9832), (-103.2548,19.0169), (-103.1524,18.9210),
(-103.0156,19.0467), (-103.0450,19.1273), (-102.7738,19.2532),

(-102.6882,19.2090), (-102.5595,19.4305), (-102.5984,19.5311),
(-102.7577,19.4955), (-102.7352,19.6073), (-102.8267,19.6806),

(-102.8408,19.7858), (-102.7591,19.8244), (-102.7508,19.9098),
(-102.7973,19.9717), (-103.0604,19.9796), (-103.0580,20.1515),

(-102.8897,20.1782), (-102.8375,20.1205), (-102.4613,20.3576),
(-102.0213,20.4147), (-102.0207,20.3478), (-101.9485,20.3613),

(-101.9205,20.2110), (-101.6968,20.2058), (-101.6221,20.3332),

(-101.4670,20.3502), (-101.3446,20.2519), (-101.4132,20.2179),

(-101.3946,20.0480), (-101.1920,20.0197), (-101.1637,20.1114),
(-101.0287,20.1061), (-100.9562,19.9409), (-100.8057,19.9902),

(-100.7287,19.9233), (-100.5724,20.0038), (-100.4953,19.9295),
(-100.4401,19.9994), (-100.3351,20.0020), (-100.3701,20.1112),

(-100.3177,20.3163), (-100.1497,20.2253), (-100.2052,20.1586),
(-100.0292,19.8652), (-100.0990,19.8614), (-100.1161,19.7351),

(-100.2444,19.6780), (-100.1586,19.4408), (-100.2835,19.3619),
(-100.3840,19.0572), (-100.5054,19.0018), (-100.6749,18.7823),

(-100.7283,18.8576), (-100.7725,18.8231), (-100.7240,18.5007),
(-100.5649,18.3927), (-100.6208,18.3254), (-100.7791,18.4624),

(-100.9415,18.4256), (-100.9972,18.5104), (-101.2726,18.5279),
(-101.5025,18.4688), (-101.6219,18.5834), (-101.7722,18.5881),

(-101.8848,18.5172), (-101.8654,18.2392), (-102.1700,18.1397),
(-102.1928,17.9974), (-102.1406,17.9221), (-102.1942,17.8855),

(-103.4368,18.2645),

[

(-103.9450,22.7787), (-103.7596,22.7436), (-103.9372,22.3564),
(-103.8728,22.1879), (-103.7337,22.6034), (-103.6074,22.5857),

(-103.5629,22.4547), (-103.6475,22.3654), (-103.5994,22.2963),
(-103.6901,22.1291), (-103.4409,22.1377), (-103.4097,22.2374),

(-103.2936,22.2761), (-103.3666,22.3758), (-103.2835,22.4403),
(-103.1645,22.4104), (-103.2176,22.3038), (-103.0491,22.2699),

(-103.1007,22.2129), (-103.0711,22.1457), (-103.1484,22.1071),
(-103.1063,22.0323), (-103.2767,22.0072), (-103.4725,21.8275),

(-103.5766,21.8173), (-103.5112,21.7523), (-103.5487,21.5971),
(-103.6962,21.5611), (-103.7081,21.4733), (-103.5998,21.3964),

(-103.5420,21.4514), (-103.5066,21.3821), (-103.7278,21.3118),
(-103.6392,21.2857), (-103.6732,21.2181), (-103.5729,21.1168),

(-103.4297,21.1474), (-103.3527,21.0439), (-103.2064,21.0938),
(-103.1224,21.0523), (-103.0502,21.0990), (-103.0930,21.1288),

(-103.0594,21.2586), (-102.9014,21.2071), (-102.7163,21.3928),
(-102.6639,21.3454), (-102.6159,21.5424), (-102.7940,21.6824),

(-102.6549,21.7771), (-102.2889,21.6369), (-102.1175,21.7172),
(-102.0118,21.8680), (-101.8073,21.9232), (-101.8834,22.0152),

(-101.7700,22.0493), (-101.6163,21.9236), (-101.5338,21.9331),
(-101.4733,21.8128), (-101.5500,21.7921), (-101.6067,21.5349),

(-101.5344,21.4246), (-101.6437,21.4008), (-101.6086,21.3030),
(-101.7275,21.2707), (-101.9103,20.9270), (-102.0540,20.8234),

(-101.9227,20.5973), (-102.0449,20.3949), (-102.4881,20.2999),
(-102.8052,20.0859), (-102.9315,20.1528), (-102.9175,20.1099),

(-103.0376,20.1043), (-103.0183,19.9374), (-102.9428,19.9742),
(-102.7184,19.8752), (-102.7268,19.7899), (-102.8085,19.7512),

(-102.7943,19.6461), (-102.7028,19.5727), (-102.7254,19.4610),
(-102.5661,19.4965), (-102.5270,19.3973), (-102.6559,19.1744),

(-102.7415,19.2186), (-103.0127,19.0927), (-102.9832,19.0122),
(-103.1201,18.8865), (-103.2209,18.9823), (-103.4975,18.9278),

(-103.4887,19.3024), (-103.6232,19.4805), (-103.8282,19.3746),
(-104.0755,19.4654), (-104.1380,19.2831), (-104.5535,19.2227),

(-104.6142,19.1108), (-104.8377,19.1956), (-104.8226,19.2589),
(-104.9850,19.2861), (-105.1389,19.5473), (-105.3022,19.6506),

(-105.7392,20.3737), (-105.6488,20.4630), (-105.3555,20.4995),
(-105.1780,20.8819), (-104.9992,20.9054), (-104.8674,21.0181),

(-104.5806,20.9204), (-104.2713,20.5932), (-104.3011,20.7769),
(-104.3960,20.7775), (-104.2233,20.9328), (-104.2364,21.1779),

(-103.8283,21.2511), (-103.7279,21.4317), (-103.9302,21.5913),
(-103.9345,21.7850), (-104.1776,21.8238), (-104.2098,22.0029),

(-104.3907,21.9934), (-104.3169,22.3725), (-104.2377,22.4472),
(-104.1805,22.4172), (-104.2317,22.5116), (-104.0913,22.5601),

(-104.1086,22.3799), (-103.9406,22.5024), (-104.0877,22.7286),
(-103.9450,22.7787),

],

[

(-91.9668,17.9297), (-91.9368,17.8802), (-91.8008,17.9175), (-91.7716,17.7982),
(-91.8210,17.7525), (-91.6416,17.6898),

(-91.6207,17.5454), (-91.6875,17.4765), (-91.4020,17.3816), (-91.4097,17.2210),
(-91.2455,17.1736), (-91.0407,16.9010),

(-90.9380,16.8996), (-90.8939,16.8184), (-90.6833,16.7225), (-90.5989,16.4698),

(-90.4437,16.4506), (-90.3315,16.3533),

(-90.4175,16.2337), (-90.4034,16.0557), (-91.7197,16.0556), (-92.2080,15.2265),
(-92.0541,15.0318), (-92.1475,14.9467),

(-92.1424,14.6168), (-92.2250,14.4828), (-93.3527,15.5630), (-93.7777,15.8745),
(-94.0704,16.0026), (-94.1455,16.1628),

(-94.0673,16.2615), (-94.1757,16.4511), (-93.8919,17.1671), (-93.8280,17.2468),
(-93.6897,17.2615), (-93.5076,17.5609),

(-93.3771,17.6611), (-93.4121,17.7889), (-93.2976,18.0051), (-93.1069,17.9156),
(-93.1317,17.8387), (-93.0397,17.8420),

(-93.0551,17.6221), (-92.9608,17.4925), (-92.8419,17.4905), (-92.8651,17.4195),
(-92.7952,17.4259), (-92.7630,17.3502),

(-92.5766,17.5607), (-92.4038,17.5897), (-92.3506,17.7169), (-92.1272,17.7815),
(-92.1432,17.8310), (-91.9936,17.9623),

(-91.9668,17.9297),

],

[

(-92.4503,18.6130), (-92.3528,18.4549), (-92.1699,18.4706), (-92.1543,18.1617),
(-91.9563,18.0195), (-91.6218,17.8746),

(-91.5919,18.1036), (-91.4898,18.1777), (-91.4843,18.1220), (-91.1920,17.9777),
(-90.9579,17.9650), (-90.9574,17.2393),

(-91.4187,17.2392), (-91.3616,17.3006), (-91.3985,17.3648), (-91.6844,17.4601),
(-91.6172,17.5285), (-91.6381,17.6729),

(-91.8175,17.7356), (-91.7681,17.7814), (-91.7964,17.8994), (-91.9376,17.8638),
(-91.9858,17.9447), (-92.1397,17.8142),

(-92.1237,17.7646), (-92.3343,17.7095), (-92.3479,17.6135), (-92.5731,17.5438),
(-92.7595,17.3334), (-92.7917,17.4091),

(-92.8616,17.4026), (-92.8384,17.4737), (-92.9573,17.4756), (-93.0516,17.6052),
(-93.0362,17.8251), (-93.1282,17.8219),

(-93.1034,17.8987), (-93.2941,17.9882), (-93.4086,17.7721), (-93.3736,17.6442),
(-93.6502,17.3023), (-93.7190,17.3788),

(-93.6378,17.5494), (-93.7635,17.6762), (-93.9691,17.7514), (-93.9974,17.8454),
(-94.1153,17.8705), (-94.1585,18.2238),

(-93.4542,18.4397), (-92.8984,18.4570), (-92.6913,18.6350), (-92.4684,18.6672),
(-92.4503,18.6130),

],

[

(-96.6791,18.6946), (-96.4497,18.5942), (-96.2051,18.1776), (-95.8131,18.1440),
(-95.7792,17.9614), (-95.8841,17.7397),

(-95.6796,17.5213), (-95.2070,17.7464), (-95.1552,17.6561), (-95.2477,17.6179),
(-95.0414,17.3727), (-94.8519,17.3096),

(-94.8846,17.1900), (-93.8081,17.1321), (-94.0852,16.4434), (-93.9769,16.2538),
(-94.0481,16.1177), (-93.9435,15.9755),

(-94.4942,16.1681), (-95.1085,16.1648), (-95.4076,15.9480), (-95.9364,15.7986),
(-96.2256,15.6485), (-96.5513,15.6227),

(-97.2210,15.8945), (-97.8076,15.9544), (-98.2317,16.2055), (-98.5870,16.2946),
(-98.3802,16.4257), (-98.4263,16.5301),

(-98.2307,16.5713), (-98.2716,16.6830), (-98.1336,16.7485), (-98.1368,16.9504),
(-98.0307,17.0333), (-98.2504,17.1169),

(-98.2305,17.2130), (-98.3533,17.2738), (-98.3593,17.5360), (-98.5192,17.6870),
(-98.4567,17.8757), (-98.3113,17.9462),

(-98.2372,17.8897), (-98.1563,18.0002), (-97.8390,17.9254), (-97.7150,18.0315),

(-97.7904,18.0821), (-97.8628,18.0084),

(-97.8302,18.0814), (-97.8881,18.1598), (-97.8165,18.3012), (-97.6711,18.3113),
(-97.6695,18.1911), (-97.5023,18.0385),

(-97.3128,18.1746), (-97.1740,18.2173), (-97.0462,18.1721), (-96.9946,18.2477),
(-96.8552,18.2584), (-96.7530,18.4124),

(-96.6714,18.4014), (-96.7247,18.5528), (-96.6791,18.6946),

],

[

(-101.3596,21.8581), (-100.9842,21.7720), (-100.8336,21.6812),
(-100.8574,21.6348), (-100.6098,21.5193), (-100.4529,21.5596),

(-100.4263,21.6998), (-100.2601,21.7045), (-100.0461,21.5372),
(-99.9385,21.5401), (-99.9237,21.4620), (-99.7542,21.5078),

(-99.7623,21.3300), (-99.6473,21.3106), (-99.6642,21.2337), (-99.8096,21.1641),
(-99.9687,21.2261), (-100.0480,20.9384),

(-100.2515,20.9738), (-100.4042,20.8886), (-100.4642,20.9304),
(-100.5717,20.8164), (-100.5425,20.7345), (-100.5910,20.7150),

(-100.4887,20.6119), (-100.4683,20.3808), (-100.3148,20.2496),
(-100.3878,20.0823), (-100.3528,19.9731), (-100.4577,19.9705),

(-100.5129,19.9006), (-100.5708,19.9763), (-100.7463,19.8944),
(-100.8234,19.9613), (-100.9725,19.9112), (-101.0243,20.0721),

(-101.1813,20.0825), (-101.2097,19.9908), (-101.4123,20.0191),
(-101.4309,20.1890), (-101.3622,20.2230), (-101.4676,20.2559),

(-101.4768,20.3190), (-101.6397,20.3043), (-101.7145,20.1769),
(-101.9084,20.1756), (-101.9668,20.3329), (-102.1209,20.3753),

(-101.9727,20.6030), (-102.1040,20.8290), (-101.9603,20.9326),
(-101.7775,21.2764), (-101.6586,21.3087), (-101.6937,21.4064),

(-101.5844,21.4303), (-101.6566,21.5361), (-101.6000,21.7978),
(-101.3596,21.8581),

],

[

(-102.2880,22.4228), (-102.1473,22.3521), (-102.1507,22.2942),
(-102.0324,22.3029), (-101.9954,22.1695), (-102.0405,22.1548),

(-101.8467,22.0454), (-101.9053,21.9954), (-101.8292,21.9035),

(-102.0336,21.8482), (-102.1394,21.6975), (-102.3267,21.6127),

(-102.6406,21.7607), (-102.7856,21.7273), (-102.8861,21.8478),
(-102.6991,22.1067), (-102.6765,22.2957), (-102.5112,22.2977),

(-102.3222,22.3943), (-102.3125,22.4640), (-102.2880,22.4228),

],

[

(-99.1745,21.6856), (-99.0382,21.1653), (-99.3879,21.1063), (-99.3810,20.9536),
(-99.5420,20.7342), (-99.4945,20.6544),

(-99.8272,20.5362), (-99.8589,20.2561), (-99.9205,20.2696), (-99.9194,20.1983),
(-99.9872,20.2524), (-99.9274,20.0479),

(-100.0932,19.9976), (-100.2096,20.0605), (-100.2429,20.1286),
(-100.1873,20.1953), (-100.4883,20.3797), (-100.5087,20.6108),

(-100.6110,20.7139), (-100.5625,20.7334), (-100.5917,20.8153),
(-100.4843,20.9293), (-100.4242,20.8875), (-100.2715,20.9727),

(-100.0680,20.9373), (-99.9887,21.2250), (-99.8305,21.1628), (-99.6842,21.2326),
(-99.6673,21.3095), (-99.7556,21.3003),

(-99.7930,21.3668), (-99.7443,21.5922), (-99.6553,21.5816), (-99.5455,21.4392),
(-99.3966,21.4391), (-99.3660,21.5612),

(-99.2942,21.5459), (-99.1745,21.6856),

],

[

(-100.6103,24.4384), (-100.6404,24.2596), (-100.5219,24.1718),
(-100.5560,23.8681), (-100.4508,23.7195), (-100.5223,23.4385),

(-100.4609,23.3828), (-100.4842,23.2073), (-100.3331,23.1750),
(-100.3765,23.2797), (-100.2787,23.3256), (-100.1006,23.2452),

(-100.1126,23.1376), (-100.0062,23.0743), (-100.1078,22.9843),
(-100.0247,22.8300), (-100.0907,22.7554), (-99.8748,22.7539),

(-99.8050,22.6630), (-99.6709,22.6838), (-99.5201,22.6141), (-99.4791,22.6224),
(-99.5299,22.7481), (-99.4814,22.7525),

(-99.3109,22.6367), (-99.2163,22.4102), (-98.8625,22.3513), (-98.6449,22.4062),
(-98.2872,22.2370), (-98.3954,22.1199),

(-98.4329,21.9530), (-98.5451,21.9545), (-98.4679,21.9244), (-98.5137,21.8464),
(-98.4185,21.7508), (-98.6112,21.6134),

(-98.4681,21.3683), (-98.6520,21.3179), (-98.5621,21.2593), (-98.5851,21.1672),
(-98.7923,21.1327), (-98.9186,21.2866),

(-99.0811,21.2645), (-99.1056,21.5707), (-99.1667,21.6525), (-99.2863,21.5126),
(-99.3581,21.5280), (-99.3875,21.4066),

(-99.4886,21.3960), (-99.7364,21.5590), (-99.7823,21.4534), (-99.9359,21.4277),
(-99.9507,21.5058), (-100.0582,21.5028),

(-100.2723,21.6702), (-100.4385,21.6654), (-100.4650,21.5253),
(-100.6220,21.4850), (-100.8695,21.6004), (-100.8457,21.6469),

(-100.9963,21.7377), (-101.3406,21.8211), (-101.5210,21.7952),
(-101.3478,21.9888), (-101.4051,22.0102), (-101.3455,22.2032),

(-101.4585,22.3374), (-101.3062,22.5071), (-101.4023,22.6784),
(-101.3709,22.7151), (-101.5155,22.7468), (-101.6878,22.5045),

(-101.8137,22.4634), (-101.9051,22.5634), (-101.8851,22.6582),
(-101.9733,22.6477), (-102.2307,22.8713), (-102.3000,23.0559),

(-102.2532,23.1430), (-102.3299,23.2454), (-102.2776,23.4649),
(-102.1597,23.3700), (-101.8053,23.4657), (-101.2111,23.9460),

(-101.2358,24.1920), (-101.0814,24.2402), (-101.0018,24.4702),
(-101.0215,24.3716), (-100.9589,24.3159), (-100.7201,24.5304),

(-100.6103,24.4384),

],

[

(-98.0530,19.7268), (-98.0400,19.6435), (-97.8644,19.5609), (-97.8564,19.4558),
(-97.7873,19.4989), (-97.6227,19.3130),

(-97.8329,19.2979), (-97.9237,19.1580), (-98.0292,19.2261), (-98.1664,19.0977),
(-98.3465,19.1720), (-98.5306,19.4606),

(-98.6632,19.4658), (-98.7195,19.5491), (-98.5746,19.6445), (-98.3862,19.6121),
(-98.3352,19.7273), (-98.1658,19.6713),

(-98.0530,19.7268),

],

[

(-97.8422,20.8732), (-97.6923,20.8326), (-97.7477,20.6727), (-97.5350,20.5161),
(-97.6415,20.4330), (-97.7325,20.4769),

(-97.7641,20.4054), (-97.6698,20.3599), (-97.7556,20.3012), (-97.7293,20.2015),
(-97.5998,20.1868), (-97.5750,20.0971),

(-97.3714,20.2714), (-97.1080,20.1560), (-97.2938,19.8847), (-97.2932,19.6811),
(-97.4261,19.5972), (-97.3384,19.4909),

(-97.3885,19.4240), (-97.3508,19.3801), (-96.9684,19.2815), (-97.0405,19.1982),
(-97.0108,19.1324), (-97.1817,19.1718),

(-97.2498,19.0951), (-97.2298,18.8967), (-97.3433,18.7837), (-97.3340,18.6760),
(-97.1274,18.5849), (-97.0647,18.4401),

(-96.8804,18.5362), (-96.7024,18.3921), (-96.8350,18.2226), (-96.9745,18.2119),
(-97.0261,18.1362), (-97.1539,18.1815),

(-97.2927,18.1388), (-97.4594,18.0040), (-97.5380,18.0107), (-97.6494,18.1553),
(-97.6510,18.2755), (-97.7964,18.2654),

(-97.8680,18.1240), (-97.8101,18.0456), (-97.8426,17.9725), (-97.7703,18.0463),
(-97.6949,17.9957), (-97.8188,17.8896),

(-98.1362,17.9644), (-98.2171,17.8539), (-98.2911,17.9104), (-98.4324,17.8375),
(-98.4603,17.9671), (-98.6667,17.9084),

(-98.8344,17.9783), (-98.7530,18.0225), (-98.8904,18.0349), (-98.9027,18.1460),
(-99.0232,18.1706), (-99.0951,18.3099),

(-98.8475,18.5109), (-98.7240,18.4306), (-98.7717,18.7344), (-98.6354,19.0323),
(-98.6695,19.4719), (-98.5368,19.4666),

(-98.3527,19.1780), (-98.1727,19.1037), (-98.0355,19.2321), (-97.9299,19.1640),
(-97.8392,19.3039), (-97.6221,19.3270),

(-97.7936,19.5049), (-97.8627,19.4618), (-97.8707,19.5669), (-98.0462,19.6496),
(-98.0652,19.7400), (-98.1062,19.6780),

(-98.2491,19.6859), (-98.2014,19.7772), (-98.3245,19.8688), (-98.1668,19.9942),
(-98.1877,20.0645), (-98.1030,20.1648),

(-98.1826,20.2759), (-98.2791,20.2367), (-98.2933,20.2846), (-98.1325,20.3595),
(-97.9111,20.6147), (-97.9241,20.7474),

(-97.8422,20.8732),

],

[

(-99.0190,19.9754), (-99.1672,20.0185), (-99.2383,19.8633), (-99.3962,19.7576),
(-99.3862,19.8393), (-99.4991,19.8941),

(-99.4510,19.9921), (-99.5279,20.1694), (-99.7392,20.1613), (-99.8822,20.2607),
(-99.8501,20.5446), (-99.5174,20.6629),

(-99.5649,20.7427), (-99.4039,20.9621), (-99.4107,21.1149), (-99.0680,21.1681),
(-99.0617,21.2990), (-98.9493,21.3283),

(-98.8858,21.1940), (-98.6960,21.1755), (-98.6064,21.2162), (-98.5926,21.2997),
(-98.6802,21.3639), (-98.4758,21.4164),

(-98.5403,21.2800), (-98.4102,21.1689), (-98.2702,21.2074), (-98.2859,21.1609),
(-98.1358,21.0889), (-98.2563,20.8746),

(-98.2238,20.8043), (-98.3206,20.7905), (-98.3375,20.8729), (-98.3964,20.8663),
(-98.5466,20.6940), (-98.4153,20.7499),

(-98.4752,20.5916), (-98.5947,20.4965), (-98.5454,20.3583), (-98.4551,20.3289),
(-98.3129,20.4171), (-98.0619,20.6809),

(-97.9702,20.5185), (-98.1148,20.3337), (-98.2756,20.2588), (-98.2614,20.2109),
(-98.1649,20.2501), (-98.0853,20.1391),

(-98.1700,20.0387), (-98.1491,19.9685), (-98.3068,19.8430), (-98.1851,19.7272),
(-98.3238,19.7075), (-98.3748,19.5923),

(-98.6619,19.5876), (-98.5943,19.7519), (-98.6716,19.8509), (-98.8863,19.8903),
(-98.9600,19.7856), (-99.0263,19.8522),

(-98.9581,19.8771), (-98.9942,20.0604), (-99.0190,19.9754),

],

[

(-98.2334,22.5173), (-98.0962,22.3980), (-98.0102,22.4011), (-98.0207,22.3466),
(-97.9133,22.2701), (-97.7903,22.3125),

(-97.7124,21.9989), (-97.3349,21.6018), (-97.4291,21.2588), (-97.1814,20.6786),
(-96.4389,19.8494), (-96.2887,19.3236),

(-96.1064,19.2014), (-96.0647,19.0625), (-95.9527,19.0410), (-95.9072,18.8677),
(-95.6194,18.7139), (-95.1532,18.6872),

(-94.9949,18.5470), (-94.7733,18.5126), (-94.4910,18.1277), (-94.0730,18.1804),
(-94.0255,17.8305), (-93.9076,17.8054),

(-93.8793,17.7115), (-93.6737,17.6362), (-93.5463,17.5068), (-93.6292,17.3388),
(-93.5405,17.2645), (-93.5964,17.2047),

(-93.7379,17.1884), (-93.8052,17.0830), (-94.8817,17.1409), (-94.8490,17.2605),
(-95.0385,17.3237), (-95.2448,17.5688),

(-95.1523,17.6070), (-95.2041,17.6973), (-95.5575,17.4890), (-95.6766,17.4721),
(-95.7841,17.5469), (-95.7572,17.5949),

(-95.8813,17.6920), (-95.7763,17.9123), (-95.8102,18.0949), (-96.2022,18.1285),

(-96.4507,18.5478), (-96.6771,18.6463),

(-96.6906,18.3353), (-96.7934,18.4854), (-96.8977,18.5229), (-97.0820,18.4269),
(-97.1447,18.5716), (-97.3513,18.6627),

(-97.3605,18.7704), (-97.2471,18.8834), (-97.2670,19.0818), (-97.1989,19.1585),
(-97.0280,19.1192), (-97.0577,19.1849),

(-96.9857,19.2682), (-97.3681,19.3668), (-97.4057,19.4107), (-97.3556,19.4776),
(-97.4434,19.5839), (-97.3104,19.6678),

(-97.3111,19.8714), (-97.1253,20.1427), (-97.4546,20.2543), (-97.5982,20.0842),
(-97.6170,20.1735), (-97.7545,20.1963),

(-97.7729,20.2879), (-97.6870,20.3466), (-97.7814,20.3921), (-97.7497,20.4636),
(-97.6587,20.4197), (-97.5523,20.5028),

(-97.6001,20.5755), (-97.7164,20.5866), (-97.7731,20.7106), (-97.7095,20.8193),
(-97.7889,20.8470), (-97.8971,20.8542),

(-97.9135,20.6384), (-97.9993,20.5285), (-98.0968,20.6933), (-98.1816,20.5567),
(-98.4917,20.3411), (-98.5800,20.3702),

(-98.6296,20.5089), (-98.5102,20.6040), (-98.4525,20.7659), (-98.5816,20.7064),
(-98.4791,20.7696), (-98.4597,20.8679),

(-98.2587,20.8168), (-98.2912,20.8871), (-98.1707,21.1033), (-98.3082,21.1624),
(-98.3051,21.2199), (-98.4452,21.1813),

(-98.5753,21.2924), (-98.4985,21.4180), (-98.6769,21.6676), (-98.4842,21.8050),
(-98.5794,21.9006), (-98.5336,21.9786),

(-98.6108,22.0087), (-98.4986,22.0072), (-98.4611,22.1741), (-98.3529,22.2912),
(-98.6113,22.4286), (-98.7156,22.4223),

(-98.4663,22.4855), (-98.3443,22.4421), (-98.3436,22.5212), (-98.2334,22.5173),

],

[

(-100.0155,27.6766), (-99.8140,27.8231), (-99.7218,27.7239), (-99.9240,27.5914),
(-99.9166,27.5326), (-99.7547,27.4769),

(-99.6421,27.0984), (-99.7611,26.9189), (-99.5808,26.8896), (-99.6514,26.6950),
(-99.3838,26.6197), (-99.3883,26.3221),

(-99.1809,26.2751), (-99.1428,26.0740), (-99.0034,26.1174), (-98.8778,25.9789),
(-98.7988,26.0598), (-98.5559,26.0295),

(-98.5525,25.4904), (-98.3891,25.4904), (-98.3907,25.4477), (-98.9119,25.0654),
(-99.0326,25.1209), (-99.1429,25.0476),

(-99.1647,24.7628), (-99.2799,24.7914), (-99.5362,24.5966), (-99.7267,24.5363),
(-99.5808,24.3753), (-99.6493,24.0916),

(-99.4451,23.8346), (-99.5933,23.7293), (-99.8569,23.7412), (-99.9595,23.5192),
(-99.9187,23.3380), (-100.0809,23.3549),

(-100.1469,23.2975), (-100.0787,23.2633), (-100.1030,23.1869),
(-100.2319,23.2067), (-100.2217,23.2676), (-100.3809,23.2218),

(-100.3375,23.1172), (-100.4926,23.1642), (-100.4653,23.3250),
(-100.5266,23.3807), (-100.4537,23.6421), (-100.5727,23.8893),

(-100.5262,24.1139), (-100.6448,24.2017), (-100.6147,24.3805),
(-100.8150,24.5385), (-100.8417,24.8229), (-100.7411,24.9083),

(-100.8370,24.9879), (-100.8183,25.1598), (-100.7221,25.2324),
(-100.2332,25.2135), (-100.3183,25.3343), (-100.4786,25.3358),

(-100.4600,25.4102), (-100.6305,25.5070), (-100.5483,25.5337),
(-100.7982,25.6437), (-100.8782,25.8261), (-100.8502,25.9552),

(-100.9227,25.9673), (-100.9338,26.0883), (-101.2300,26.3884),
(-100.7672,26.7611), (-100.7041,26.6373), (-100.5480,26.8359),

(-100.5437,27.0777), (-100.6954,27.1448), (-100.8145,27.0780),
(-100.8499,27.2303), (-100.6906,27.3727), (-100.4251,27.4327),

(-100.3567,27.7295), (-100.1913,27.8463), (-100.0155,27.6766),

],

[

(-102.3220,29.9379), (-102.0786,29.8458), (-101.3943,29.8295),
(-101.2933,29.6310), (-101.2391,29.6787), (-101.2488,29.5763),

(-101.0480,29.5169), (-100.6546,29.1488), (-100.4752,28.6983),
(-100.3055,28.5327), (-100.2645,28.3078), (-99.8948,28.0036),

(-99.8062,27.7974), (-99.9638,27.6310), (-100.1572,27.8185), (-100.3226,27.7017),
(-100.3910,27.4049), (-100.6566,27.3449),

(-100.8158,27.2025), (-100.7804,27.0502), (-100.6614,27.1170),
(-100.5097,27.0499), (-100.5139,26.8081), (-100.6700,26.6095),

(-100.7331,26.7333), (-101.1959,26.3606), (-100.8997,26.0605),
(-100.8886,25.9395), (-100.8161,25.9274), (-100.8441,25.7983),

(-100.7641,25.6159), (-100.5142,25.5059), (-100.5965,25.4792),
(-100.4259,25.3824), (-100.4445,25.3080), (-100.2843,25.3065),

(-100.1992,25.1857), (-100.3036,25.1534), (-100.5726,25.2233),
(-100.7842,25.1320), (-100.8029,24.9601), (-100.7070,24.8805),

(-100.8076,24.7951), (-100.7916,24.5060), (-101.0671,24.5464),
(-101.1907,24.7242), (-101.3340,24.7730), (-101.4559,24.6949),

(-101.6197,24.7176), (-101.5983,24.7957), (-101.7739,24.8365),
(-101.8508,24.9814), (-102.3064,25.0912), (-102.4776,25.0816),

(-102.4607,24.9629), (-102.6881,25.0314), (-102.8920,24.7003),
(-103.1198,24.7827), (-103.5461,25.2488), (-103.4963,25.3473),

(-103.3686,25.3421), (-103.5293,25.5119), (-103.3712,25.6852),
(-103.3825,26.1287), (-103.2910,26.2738), (-103.4067,26.5978),

(-103.6818,26.6874), (-103.8937,26.8975), (-103.7806,27.0264),
(-103.9194,27.3212), (-103.8967,27.5638), (-103.9991,27.9733),

(-103.9357,27.9357), (-103.9702,27.9942), (-103.6333,28.6757),
(-103.5498,28.7017), (-103.3388,29.0468), (-103.1432,29.0275),

(-103.0202,29.2267), (-102.8897,29.2716), (-102.9067,29.3984),
(-102.6934,29.8024), (-102.4003,29.8202), (-102.3220,29.9379),

],

[

(-99.7297,27.7191), (-99.5195,27.6204), (-99.4860,27.5321), (-99.5466,27.3657),

(-99.4483,27.2988), (-99.4523,27.0653),

(-99.2711,26.8824), (-99.0833,26.4285), (-98.7984,26.3979), (-98.6616,26.2647),
(-98.4307,26.2516), (-98.1752,26.0771),

(-97.6335,26.0618), (-97.3906,25.8602), (-97.2473,25.9815), (-97.1070,25.9769),
(-97.1407,25.7064), (-97.4757,25.0703),

(-97.6577,24.4482), (-97.7389,22.8556), (-97.8342,22.5270), (-97.7569,22.2108),
(-97.8195,22.1541), (-98.1766,22.4219),

(-98.3008,22.4209), (-98.3028,22.3415), (-98.4928,22.3810), (-98.8855,22.3052),
(-99.2392,22.3641), (-99.3339,22.5905),

(-99.5044,22.7064), (-99.5529,22.7020), (-99.5021,22.5762), (-99.5431,22.5680),
(-99.6939,22.6377), (-99.8280,22.6169),

(-99.8977,22.7078), (-100.1216,22.7210), (-100.0476,22.7838),
(-100.1307,22.9382), (-100.0293,23.0296), (-100.1356,23.0915),

(-100.0973,23.2751), (-100.1655,23.3093), (-100.0995,23.3666),
(-99.9373,23.3497), (-99.9781,23.5310), (-99.8755,23.7529),

(-99.6119,23.7411), (-99.4637,23.8463), (-99.6679,24.1033), (-99.5994,24.3870),
(-99.7453,24.5480), (-99.5548,24.6083),

(-99.2985,24.8032), (-99.1833,24.7745), (-99.1615,25.0593), (-99.0512,25.1326),
(-98.9305,25.0772), (-98.4093,25.4594),

(-98.4077,25.5021), (-98.5711,25.5021), (-98.5745,26.0412), (-98.8174,26.0716),
(-98.8964,25.9906), (-99.0220,26.1291),

(-99.1610,26.0855), (-99.1995,26.2868), (-99.4372,26.3813), (-99.4024,26.6315),
(-99.6700,26.7067), (-99.5901,26.8697),

(-99.6723,26.9418), (-99.7797,26.9306), (-99.6607,27.1101), (-99.7733,27.4887),
(-99.9352,27.5443), (-99.9426,27.6031),

(-99.7297,27.7191),

],

[

(-87.9391,21.6325), (-87.5025,21.5102), (-87.5007,21.0098), (-87.7161,20.6570),
(-88.1161,20.2772), (-88.3942,20.2473),

(-88.5072,20.1863), (-88.4947,20.1364), (-88.6813,20.0818), (-88.8194,19.8677),
(-89.0226,19.7624), (-89.0467,19.7061),

(-88.9944,19.6892), (-89.1228,19.6877), (-89.1081,19.5642), (-89.1899,19.6271),
(-89.2994,19.5324), (-89.4746,19.6843),

(-89.6076,19.9883), (-89.7604,20.1730), (-89.8068,20.1291), (-90.0212,20.4869),
(-90.0855,20.4239), (-90.2069,20.4358),

(-90.2507,20.5446), (-90.4113,20.5569), (-90.4321,20.8558), (-90.3639,21.0138),
(-90.1134,21.1688), (-89.6800,21.3010),

(-89.6919,21.3660), (-89.6746,21.3044), (-88.8343,21.4160), (-88.5957,21.5458),
(-87.9391,21.6325),

],

[

(-90.5724,19.8423), (-90.4457,19.9773), (-90.5018,20.5264), (-90.3764,20.8839),
(-90.3838,20.5853), (-90.2233,20.5731),

(-90.1795,20.4643), (-90.0580,20.4524), (-89.9937,20.5154), (-89.7793,20.1575),
(-89.7329,20.2015), (-89.5801,20.0168),

(-89.4562,19.7239), (-89.1190,19.4310), (-89.0931,18.3722), (-89.1154,18.1274),
(-89.1737,18.1262), (-89.1171,18.0375),

(-89.1810,17.9711), (-89.1247,17.9313), (-89.1242,17.7906), (-90.9968,17.7904),
(-90.9972,17.9404), (-91.2313,17.9531),

(-91.5236,18.0974), (-91.5290,18.1531), (-91.6311,18.0790), (-91.6610,17.8501),

(-92.1535,18.0824), (-92.2091,18.4460),

(-92.4506,18.4740), (-92.5076,18.6426), (-92.0106,18.6918), (-91.8822,18.6022),
(-90.9175,19.1782), (-90.7267,19.3650),

(-90.7161,19.6837), (-90.5724,19.8423),

],

[

(-87.0677,21.6258), (-86.9557,21.5831), (-86.8002,21.2734), (-86.7721,21.4408),
(-86.7804,21.1920), (-86.7157,21.1622),

(-86.8558,20.8628), (-87.2682,20.4595), (-87.4123,20.2353), (-87.4593,19.7809),
(-87.4570,19.9485), (-87.4488,19.8601),

(-87.5708,19.7907), (-87.6604,19.6134), (-87.6424,19.6864), (-87.7374,19.6615),
(-87.6632,19.4956), (-87.5165,19.5359),

(-87.4438,19.6455), (-87.3991,19.5884), (-87.4293,19.4630), (-87.6290,19.3655),
(-87.6629,19.2855), (-87.6308,19.1653),

(-87.5476,19.2866), (-87.4348,19.3025), (-87.5283,19.1967), (-87.8415,18.1593),
(-88.0312,18.1281), (-88.0313,18.3877),

(-88.4862,18.4691), (-88.8772,17.8569), (-89.0586,17.9698), (-89.2303,17.9177),
(-89.1679,18.0234), (-89.2245,18.1121),

(-89.1662,18.1133), (-89.1439,18.3581), (-89.1699,19.4169), (-89.3228,19.5468),
(-89.2133,19.6414), (-89.1315,19.5786),

(-89.1461,19.7021), (-89.0178,19.7035), (-89.0701,19.7205), (-89.0459,19.7768),
(-88.8427,19.8821), (-88.7047,20.0962),

(-88.5181,20.1508), (-88.5306,20.2007), (-88.4176,20.2617), (-88.1395,20.2916),
(-87.7394,20.6714), (-87.5241,21.0242),

(-87.5259,21.5246), (-87.2174,21.4515), (-87.0942,21.5052), (-87.0677,21.6258),

],

[

(-100.6242,18.9057), (-100.4410,18.7998), (-100.4986,18.6540),
(-100.3116,18.3752), (-100.0936,18.5335), (-100.0859,18.6006),

(-99.9627,18.5649), (-99.7907,18.6441), (-99.7336,18.7555), (-99.6672,18.7286),
(-99.6927,18.8029), (-99.6253,18.7358),

(-99.4861,18.7358), (-99.3464,18.4779), (-99.1633,18.4341), (-99.1252,18.4853),
(-99.0468,18.2574), (-98.8613,18.1704),

(-98.8490,18.0592), (-98.7116,18.0469), (-98.7930,18.0026), (-98.6254,17.9327),
(-98.3954,17.9773), (-98.4613,17.6984),

(-98.2978,17.5245), (-98.2918,17.2623), (-98.1690,17.2015), (-98.1889,17.1055),
(-97.9692,17.0218), (-98.0753,16.9389),

(-98.0721,16.7370), (-98.2101,16.6715), (-98.1692,16.5598), (-98.3648,16.5186),
(-98.3183,16.4154), (-98.3689,16.3490),

(-98.5255,16.2831), (-98.7502,16.5269), (-98.8858,16.5060), (-99.6497,16.6702),
(-99.9915,16.8883), (-101.0841,17.2522),

(-101.2294,17.4165), (-101.6839,17.6588), (-101.8562,17.8918),
(-102.0757,17.9886), (-102.1825,17.9399), (-102.2256,17.9997),

(-102.1871,18.1917), (-101.9023,18.2678), (-101.9217,18.5458),
(-101.8090,18.6167), (-101.6588,18.6120), (-101.5394,18.4974),

(-101.3095,18.5565), (-101.0340,18.5390), (-100.9784,18.4542),
(-100.8159,18.4910), (-100.6577,18.3539), (-100.6018,18.4213),

(-100.7609,18.5293), (-100.8168,18.8011), (-100.7838,18.8836),
(-100.7118,18.8109), (-100.6242,18.9057),

],

[

(-99.1109,19.5667), (-98.9365,19.2217), (-98.9624,19.0816), (-99.0584,19.0432),
(-99.2817,19.1283), (-99.3696,19.2773),

(-99.2108,19.5167), (-99.1109,19.5667),

],

[

(-99.9193,20.3021), (-99.7225,20.1813), (-99.5112,20.1894), (-99.4343,20.0121),
(-99.4824,19.9141), (-99.3694,19.8593),

(-99.3788,19.7774), (-99.2216,19.8833), (-99.1505,20.0385), (-98.9929,19.9750),
(-98.9775,20.0804), (-98.9303,19.9894),

(-98.9971,19.8357), (-98.9378,19.8064), (-98.8695,19.9103), (-98.6548,19.8709),
(-98.5775,19.7719), (-98.6913,19.5493),

(-98.6162,19.4306), (-98.6010,19.0265), (-98.7766,18.9248), (-98.9673,19.0667),
(-98.9458,19.3205), (-99.0480,19.3995),

(-99.1149,19.5954), (-99.3525,19.3036), (-99.2730,19.1251), (-99.3036,18.9488),
(-99.4267,18.8659), (-99.4925,18.7060),

(-99.6323,18.7057), (-99.6999,18.7728), (-99.6741,18.6986), (-99.7405,18.7254),
(-99.7976,18.6140), (-99.9696,18.5348),

(-100.0928,18.5705), (-100.1005,18.5034), (-100.3186,18.3451),
(-100.5055,18.6239), (-100.4480,18.7697), (-100.6339,18.8821),

(-100.4278,19.0556), (-100.3273,19.3604), (-100.2024,19.4393),
(-100.2882,19.6765), (-100.1599,19.7336), (-100.1428,19.8598),

(-100.0730,19.8637), (-100.1651,20.0655), (-100.0993,20.0261),
(-99.9336,20.0764), (-99.9949,20.2730), (-99.9255,20.2268),

(-99.9193,20.3021),

],

[

(-99.0620,19.0545), (-98.9126,19.0701), (-98.7866,18.9388), (-98.6243,19.0206),
(-98.7473,18.7426), (-98.6996,18.4388),

(-98.8230,18.5191), (-99.0648,18.3238), (-99.1426,18.4695), (-99.1802,18.4180),
(-99.3633,18.4618), (-99.5030,18.7197),

(-99.4367,18.8799), (-99.3180,18.9518), (-99.2997,19.1301), (-99.0620,19.0545),

],

[

(-108.5030,27.0830), (-108.2162,27.0748), (-108.2528,26.9461),
(-108.1640,26.9483), (-108.0238,26.7985), (-108.1012,26.6771),

(-107.9782,26.5597), (-108.0311,26.4701), (-107.8557,26.2095),
(-107.3515,26.1340), (-107.3001,25.9455), (-107.0420,25.7139),

(-106.9626,25.7141), (-106.9708,25.6444), (-107.1754,25.5699),
(-107.1401,25.5016), (-107.2123,25.4277), (-107.1215,25.2502),

(-107.1305,25.0564), (-106.6981,24.6608), (-106.5492,24.4644),
(-106.5916,24.4051), (-106.5054,24.2750), (-106.2522,24.3645),

(-106.0540,24.2571), (-105.9767,24.0265), (-105.8884,23.9797),
(-105.9223,23.9292), (-105.8327,23.8795), (-105.9307,23.8890),

(-105.9385,23.8198), (-105.8685,23.5259), (-105.7859,23.5855),
(-105.7427,23.5298), (-105.5945,23.0871), (-105.3581,23.0491),

(-105.4693,22.9200), (-105.4421,22.8107), (-105.4933,22.7325),
(-105.4044,22.5389), (-105.4281,22.4610), (-105.6130,22.5480),

(-105.6864,22.4267), (-105.8463,22.6649), (-106.4166,23.1424),

(-106.4713,23.3001), (-106.8052,23.6281), (-106.8892,23.8208),

(-108.0088,24.6403), (-108.4773,25.2640), (-109.0608,25.4494),
(-109.2680,25.6409), (-109.4441,25.6537), (-109.4946,25.9702),

(-109.3305,26.3073), (-108.9170,26.4366), (-108.9086,26.5843),
(-108.7104,26.6210), (-108.4508,27.0180), (-108.5305,27.0294),

(-108.5030,27.0830),

],

[

(-87.0000,20.5100), (-86.9750,20.5450), (-86.9000,20.5100), (-86.9250,20.4750),
(-87.0000,20.5100),

],

]

```
# Catálogo de ciudades de México con sus coordenadas reales.
```

```
# Formato: "Nombre de la ciudad": (latitud, longitud)
```

```
CATALOGO_CIUDADES = {
```

```
    "Acapulco": (16.86, -99.88),
```

```
    "Aguascalientes": (21.88, -102.30),
```

```
    "Campeche": (19.84, -90.53),
```

```
    "Cancún": (21.16, -86.85),
```

```
    "Celaya": (20.52, -100.81),
```

```
    "Chetumal": (18.50, -88.30),
```

```
    "Chihuahua": (28.64, -106.09),
```

```
    "Chilpancingo": (17.55, -99.50),
```

"Ciudad de México": (19.43, -99.13),

"Ciudad Juárez": (31.74, -106.49),

"Ciudad Obregón": (27.49, -109.94),

"Ciudad Victoria": (23.73, -99.13),

"Colima": (19.24, -103.72),

"Córdoba": (18.89, -96.93),

"Cozumel": (20.51, -86.95),

"Cuernavaca": (18.92, -99.22),

"Culiacán": (24.80, -107.39),

"Durango": (24.03, -104.67),

"Ensenada": (31.87, -116.60),

"Guadalajara": (20.67, -103.35),

"Guaymas": (27.92, -110.90),

"Hermosillo": (29.07, -110.96),

"Huatulco": (15.77, -96.12),

"Irapuato": (20.67, -101.35),

"Ixtapa-Zihuatanejo": (17.65, -101.55),

"La Paz": (24.14, -110.31),

"León": (21.12, -101.68),

"Los Cabos": (22.89, -109.91),

"Los Mochis": (25.79, -108.99),

"Manzanillo": (19.05, -104.32),

"Matamoros": (25.87, -97.50),

"Mazatlán": (23.25, -106.41),

"Mérida": (20.97, -89.62),

"Mexicali": (32.63, -115.45),

"Monclova": (26.91, -101.42),

"Monterrey": (25.69, -100.32),

"Morelia": (19.70, -101.18),

"Nuevo Laredo": (27.48, -99.51),

"Oaxaca": (17.06, -96.72),

"Pachuca": (20.12, -98.73),

"Piedras Negras": (28.70, -100.52),

"Poza Rica": (20.53, -97.45),

"Puebla": (19.04, -98.20),

"Puerto Vallarta": (20.65, -105.22),

"Querétaro": (20.59, -100.39),

"Reynosa": (26.08, -98.30),

"Salamanca": (20.57, -101.19),

"Saltillo": (25.42, -101.00),

"San Cristóbal de las Casas": (16.74, -92.64),

"San Luis Potosí": (22.15, -100.98),

"Tampico": (22.23, -97.86),

"Tapachula": (14.91, -92.26),

"Tehuacán": (18.46, -97.39),

"Tepic": (21.51, -104.89),

"Tijuana": (32.51, -117.02),

"Tlaxcala": (19.32, -98.24),

"Toluca": (19.29, -99.66),

"Torreón": (25.54, -103.44),

"Tulum": (20.21, -87.46),

"Tuxtla Gutiérrez": (16.75, -93.12),

"Uruapan": (19.41, -102.05),

"Veracruz": (19.17, -96.13),

"Villahermosa": (17.99, -92.93),

```
"Xalapa": (19.54, -96.93),

"Zacatecas": (22.77, -102.58),

}

# Ciudades precargadas al iniciar el sistema (todas son editables).

CIUDADES_INICIALES = [

    "Ciudad de México", "Guadalajara", "Monterrey", "Tijuana", "Cancún",

    "Mérida", "Veracruz", "Oaxaca", "Puebla", "León", "Puerto Vallarta",

    "La Paz", "San Luis Potosí",

]
```



```
# Rutas de vuelo de ejemplo con las que arranca el sistema.
```

```
# Formato: (origen, destino)
```

```
RUTAS_INICIALES = [
```

```
    ("Ciudad de México", "Guadalajara"),
```

```
    ("Guadalajara", "Ciudad de México"),
```

```
    ("Ciudad de México", "Monterrey"),
```

```
    ("Monterrey", "Ciudad de México"),
```

```
    ("Ciudad de México", "Cancún"),
```

```
    ("Cancún", "Ciudad de México"),
```

```
    ("Ciudad de México", "Mérida"),
```

```
    ("Mérida", "Ciudad de México"),
```

("Ciudad de México", "Veracruz"),

("Ciudad de México", "Puebla"),

("Ciudad de México", "Oaxaca"),

("Oaxaca", "Ciudad de México"),

("Ciudad de México", "San Luis Potosí"),

("Guadalajara", "Puerto Vallarta"),

("Puerto Vallarta", "Guadalajara"),

("Guadalajara", "León"),

("León", "Ciudad de México"),

("Monterrey", "Tijuana"),

("Tijuana", "Monterrey"),

```
("Monterrey", "San Luis Potosí"),
```

```
("Tijuana", "La Paz"),
```

```
("La Paz", "Tijuana"),
```

```
("Puebla", "Veracruz"),
```

```
("Veracruz", "Mérida"),
```

```
("Mérida", "Cancún"),
```

```
]
```

```
# Parámetros de simulación de vuelo
```

```
VELOCIDAD_CRUCERO_KMH = 850.0    # Velocidad media de crucero de un avión comercial
```

```
TIEMPO_TIERRA_HORAS = 0.75      # 45 minutos extra por despegue y aterrizaje
```

```
# Palabras que no se capitalizan al normalizar nombres de ciudades
```

```
PALABRAS_MENORES = {"de", "del", "la", "las", "los", "y", "el"}
```

```
# Tabla para eliminar acentos (búsquedas sin importar tildes)
```

```
_TABLA_ACENTOS = str.maketrans(
```

```
"áéíóúüñÁÉÍÓÚÜÑ", "aeiouuñAEIOUUN"
```

```
)
```

```
# -----
```

2. FUNCIONES AUXILIARES

```
def normalizar_nombre(nombre: str) → str:
```

```
    """
```

Normaliza el nombre de una ciudad: elimina espacios sobrantes y aplica

mayúsculas iniciales correctas (sin capitalizar preposiciones).

Ejemplo: "ciudad DE México" → "Ciudad de México"

```
    """
```

```
    palabras = nombre.strip().split()
```

```
if not palabras:

    return ""

resultado = []

for i, palabra in enumerate(palabras):

    palabra_low = palabra.lower()

    if palabra_low in PALABRAS_MENORES:

        resultado.append(palabra_low)

    else:

        resultado.append(palabra.capitalize())

return " ".join(resultado)
```

```
def plegar_acentos(texto: str) → str:
```

```
    """Convierte un texto a minúsculas y elimina los acentos."""
```

```
    return texto.lower().translate(_TABLA_ACENTOS)
```

```
def distancia_haversine(lat1: float, lon1: float, lat2: float, lon2: float) → float:
```

```
    """
```

```
    Calcula la distancia en kilómetros entre dos puntos geográficos
```

```
    utilizando la fórmula de Haversine.
```

```
    """
```

```
radio_tierra = 6371.0 # Radio medio de la Tierra en kilómetros
```

```
lat1_rad = math.radians(lat1)
```

```
lat2_rad = math.radians(lat2)
```

```
dlat = math.radians(lat2 - lat1)
```

```
dlon = math.radians(lon2 - lon1)
```

```
a = (math.sin(dlat / 2) ** 2
```

```
      + math.cos(lat1_rad) * math.cos(lat2_rad) * math.sin(dlon / 2) ** 2)
```

```
c = 2 * math.asin(math.sqrt(a))
```



```
return radio_tierra * c
```

```
def formato_duracion(horas: float) → str:
```

```
    """Convierte horas decimales a formato 'X h Y min'."""
```

```
    horas_enteras = int(horas)
```

```
    minutos = int(round((horas - horas_enteras) * 60))
```

```
    if minutos == 60:
```

```
        horas_enteras += 1
```

```
        minutos = 0
```

```
    return f"{horas_enteras} h {minutos:02d} min"
```

```
def formato_km(km: float) → str:
```

```
    """Formatea kilómetros con separador de miles."""
```

```
    return f"{round(km):,}".replace(",", " ")
```

```
# -----
```

```
# 3. CLASE PRINCIPAL: RED DE VUELOS
```

```
# -----
```

```
class RedVuelos:

    """

    Gestiona la red de rutas aéreas de México mediante un grafo dirigido

    (networkx.DiGraph). Cada nodo almacena su posición geográfica (lat, lon)

    y cada arista almacena su distancia en kilómetros.

    """

    def __init__(self) → None:

        self.grafo = nx.DiGraph()

        self._cargar_datos_iniciales()
```

```
# ----- Datos iniciales -----
```

```
def _cargar_datos_iniciales(self) → None:
```

```
    """Precarga las ciudades y rutas de ejemplo del sistema."""
```

```
    for nombre in CIUDADES_INICIALES:
```

```
        self._agregar_nodo(nombre)
```

```
    for origen, destino in RUTAS_INICIALES:
```

```
        self._agregar_arista(origen, destino)
```

```
def _agregar_nodo(self, nombre: str) → bool:
```

```
    """Agrega un nodo con su posición geográfica. Devuelve True si se agregó."""
```

```
if nombre not in CATALOGO_CIUDADES:
```

```
    return False
```

```
lat, lon = CATALOGO_CIUDADES[nombre]
```

```
self.grafo.add_node(nombre, pos=(lat, lon))
```

```
return True
```

```
def _agregar_arista(self, origen: str, destino: str) → bool:
```

```
    """Agrega una arista dirigida con su distancia en km. Devuelve True si se
    agregó."""
```

```
if self.grafo.has_edge(origen, destino):
```

```
    return False
```

```
lat1, lon1 = self.grafo.nodes[origen]["pos"]
```

```
lat2, lon2 = self.grafo.nodos[destino]["pos"]
```

```
km = distancia_haversine(lat1, lon1, lat2, lon2)
```

```
self.grafo.add_edge(origen, destino, km=km)
```

```
return True
```

```
# ----- Utilidades -----
```

```
def _buscar_ciudad(self, texto: str) → str | None:
```

```
"""
```

```
Busca el nombre canónico de una ciudad en el grafo, tolerando
```

```
diferencias de mayúsculas y acentos. Devuelve None si no se encuentra.
```

```
"""
```

```
texto_norm = plegar_acentos(normalizar_nombre(texto))
```

```
if not texto_norm:
```

```
    return None
```

```
candidatos = [
```

```
    n for n in self.grafo.nodes
```

```
    if plegar_acentos(n) == texto_norm
```

```
]
```

```
if len(candidatos) == 1:
```

```
    return candidatos[0]
```

```
if len(candidatos) > 1:
```

```
print("Error: El nombre es ambiguo. ¿A cuál te refieres?")
```

```
for c in candidatos:
```

```
    print(f"    - {c}")
```

```
    return None
```

```
return None
```

```
def _pedir_ciudad(self, mensaje: str) → str | None:
```

```
    """Pide el nombre de una ciudad existente al usuario y la valida."""
```

```
    texto = input(mensaje)
```

```
    ciudad = self._buscar_ciudad(texto)
```

```
    if ciudad is None:
```



```
        print(f"Error: La ciudad '{texto.strip()}' no existe. Debe agregarla  
primero.")
```

```
    return ciudad
```

```
# ----- Operaciones del menú -----
```

```
def agregar_ciudad(self, nombre: str) → None:
```

```
    """Agrega una nueva ciudad (nodo) al grafo."""
```

```
    nombre_norm = normalizar_nombre(nombre)
```

```
    if not nombre_norm:
```

```
        print("Error: El nombre de la ciudad no puede estar vacío.")
```

```
    return
```

```
if nombre_norm not in CATALOGO_CIUDADES:
```

```
    print(f"Error: '{nombre_norm}' no está en el catálogo de México.")
```

```
    return
```

```
if self.grafo.has_node(nombre_norm):
```

```
    print(f"Error: La ciudad '{nombre_norm}' ya existe en el sistema.")
```

```
    return
```

```
self._agregar_nodo(nombre_norm)
```

```
lat, lon = CATALOGO_CIUDADES[nombre_norm]
```

```
print(f"Éxito: Ciudad '{nombre_norm}' agregada ({lat:.2f}, {lon:.2f}).")
```

```
def agregar_ruta(self, origen: str, destino: str) → None:
```

```
"""Agrega una ruta de vuelo dirigida entre dos ciudades."""
```

```
origen_norm = self._buscar_ciudad(origen)
```

```
if origen_norm is None:
```

```
    print(f"Error: La ciudad de origen '{origen.strip()}' no existe.")
```

```
    return
```

```
destino_norm = self._buscar_ciudad(destino)
```

```
if destino_norm is None:
```

```
    print(f"Error: La ciudad de destino '{destino.strip()}' no existe.")
```

```
    return
```

```
if origen_norm == destino_norm:
```

```
    print("Error: El origen y el destino deben ser ciudades distintas.")
```

```
return
```

```
if self.grafo.has_edge(origen_norm, destino_norm):
```

```
    print(f"Error: La ruta de '{origen_norm}' a '{destino_norm}' ya existe.")
```

```
return
```

```
self._agregar_arista(origen_norm, destino_norm)
```

```
km = self.grafo.edges[origen_norm, destino_norm]["km"]
```

```
print(f"Éxito: Ruta de vuelo desde '{origen_norm}' hacia '{destino_norm}' "
```

```
      f"({formato_km(km)} km) agregada.")
```

```
def eliminar_ciudad(self, ciudad: str) → None:
```

```
    """Elimina una ciudad y todas sus rutas asociadas."""
```

```
ciudad_norm = self._buscar_ciudad(ciudad)
```

```
if ciudad_norm is None:
```

```
    print(f"Error: La ciudad '{ciudad.strip()}' no existe en el sistema.")
```

```
    return
```

```
self.grafo.remove_node(ciudad_norm)
```

```
print(f"Éxito: La ciudad '{ciudad_norm}' y todas sus rutas han sido eliminadas.")
```

```
def eliminar_ruta(self, origen: str, destino: str) → None:
```

```
    """Elimina una ruta de vuelo específica entre dos ciudades."""
```

```
    origen_norm = self._buscar_ciudad(origen)
```

```
    destino_norm = self._buscar_ciudad(destino)
```

```
if origen_norm is not None and destino_norm is not None:
```

```
    if self.grafo.has_edge(origen_norm, destino_norm):
```

```
        self.grafo.remove_edge(origen_norm, destino_norm)
```

```
        print(f"Éxito: Ruta de '{origen_norm}' a '{destino_norm}' eliminada.")
```

```
    return
```

```
print(f"Error: No existe una ruta directa desde '{origen.strip()}' "
```

```
      f"hacia '{destino.strip()}'.")
```

```
def ruta_mas_corta(self, origen: str, destino: str, por_km: bool = True) → list |  
None:
```

```
    """
```

```
    Calcula la ruta más corta entre dos ciudades usando el algoritmo de
```

Dijkstra. Si por_km es True minimiza la distancia en kilómetros;

en caso contrario minimiza el número de escalas.

Devuelve la lista de ciudades del camino, o None si no existe.

"""

origen_norm = self._buscar_ciudad(origen)

if origen_norm is None:

print(f"Error: La ciudad de origen '{origen.strip()}' no existe.")

return None

destino_norm = self._buscar_ciudad(destino)

if destino_norm is None:

print(f"Error: La ciudad de destino '{destino.strip()}' no existe.")

```
        return None

    if origen_norm == destino_norm:

        print("Error: El origen y el destino deben ser ciudades distintas.")

        return None

    try:

        if por_km:

            camino = nx.dijkstra_path(self.grafo, origen_norm, destino_norm,
weight="km")

        else:

            camino = nx.shortest_path(self.grafo, origen_norm, destino_norm)

    except nx.NetworkXNoPath:
```



```
print(f"Error: No existe una ruta (directa o con escalas) de "
```

```
f" '{origen_norm}' a '{destino_norm}'".")
```

```
return None
```

```
return camino
```

```
def distancia_y_duracion(self, origen: str, destino: str) → None:
```

```
    """
```

```
Muestra la distancia y el tiempo estimado de vuelo entre dos
```

```
ciudades. Si no hay vuelo directo, calcula el mejor recorrido
```

```
con escalas.
```

```
"""
```

```
origen_norm = self._buscar_ciudad(origen)
```

```
if origen_norm is None:
```

```
    print(f"Error: La ciudad de origen '{origen.strip()}' no existe.")
```

```
    return
```

```
destino_norm = self._buscar_ciudad(destino)
```

```
if destino_norm is None:
```

```
    print(f"Error: La ciudad de destino '{destino.strip()}' no existe.")
```

```
    return
```

```
if self.grafo.has_edge(origen_norm, destino_norm):
```

```
km = self.grafo.edges[origen_norm, destino_norm]["km"]
```

```
horas = km / VELOCIDAD_CRUCERO_KMH + TIEMPO_TIERRA_HORAS
```

```
print(f"Vuelo directo '{origen_norm}' → '{destino_norm}':")
```

```
print(f"  Distancia: {formato_km(km)} km")
```

```
print(f"  Duración estimada: {formato_duracion(horas)}")
```

```
return
```

```
camino = self.ruta_mas_corta(origen_norm, destino_norm, por_km=True)
```

```
if camino is None:
```

```
return
```

```

km_total = sum(

    self.grafo.edges[u, v]["km"] for u, v in zip(camino, camino[1:])

)

escalas = len(camino) - 2

horas = km_total / VELOCIDAD_CRUCERO_KMH + TIEMPO_TIERRA_HORAS * (len(camino) -
1)

print(f"No hay vuelo directo. Mejor recorrido con escalas:")

print(f" Ruta: {' → '.join(camino)} ({escalas} escala(s))")

print(f" Distancia total: {formato_km(km_total)} km")

print(f" Duración estimada: {formato_duracion(horas)}")

def listar_red(self) → None:

```

```
"""Imprime en consola todas las ciudades y rutas registradas."""

print(f"\n≡ RED ACTUAL: {self.grafo.number_of_nodes()} ciudades, "

      f"{self.grafo.number_of_edges()} rutas ≡")

print("\nCiudades:")

for nombre in sorted(self.grafo.nodes):

    lat, lon = self.grafo.nodes[nombre]["pos"]

    print(f" - {nombre} ({lat:.2f}, {lon:.2f})")

print("\nRutas de vuelo (dirigidas):")

if self.grafo.number_of_edges() == 0:

    print(" (Sin rutas registradas)")

for origen, destino in sorted(self.grafo.edges):
```

```
km = self.grafo.edges[origen, destino]["km"]
```

```
print(f" - {origen} → {destino} ({formato_km(km)} km)")
```

```
def mostrar_grafo(self, ruta_resaltada: list | None = None) → None:
```

```
    """
```

Dibuja la red de rutas aéreas sobre el mapa de México.

Si se indica una ruta_resaltada (lista de ciudades), se pinta

en rojo sobre el mapa.

```
    """
```

```
    if self.grafo.number_of_nodes() == 0:
```

```
        print("El sistema no tiene ciudades registradas para mostrar. ")
```

```
"Agregue ciudades primero.")
```

```
return
```

```
print(f"Generando mapa con {self.grafo.number_of_nodes()} ciudades y "
```

```
f"{self.grafo.number_of_edges()} rutas...")
```

```
fig, ax = plt.subplots(figsize=(12, 8))
```

```
# Fondo oceánico
```

```
ax.set_facecolor("#EAF6FF")
```

```
# Contorno del territorio mexicano (todos los anillos del país)
```

```
todos_lons, todos_lats = [], []
```

```
for anillo in FRONTERA_MEXICO:
```

```
    lons_anillo = [p[0] for p in anillo]
```

```
    lats_anillo = [p[1] for p in anillo]
```

```
    ax.fill(lons_anillo, lats_anillo, facecolor="#E7F2DC",
```

```
            edgecolor="#8FAE8B", linewidth=1.2, zorder=1)
```

```
    todos_lons.extend(lons_anillo)
```

```
    todos_lats.extend(lats_anillo)
```

```
# Posiciones geográficas de los nodos
```



```
pos = {nombre: (datos["pos"][1], datos["pos"][0]) # (lon, lat)
```

```
for nombre, datos in self.grafo.nodes(data=True)}
```

```
# Tamaño del nodo según su número de conexiones (hub)
```

```
grados = dict(self.grafo.degree())
```

```
tamano_nodos = [400 + 120 * grados[n] for n in self.grafo.nodes]
```

```
# Aristas normales (grises)
```

```
aristas_resaltadas = (set(zip(ruta_resaltada, ruta_resaltada[1:])))
```

```
if ruta_resaltada else set())
```

```
aristas_normales = [
```

```
(u, v) for u, v in self.grafo.edges

if (u, v) not in aristas_resaltadas

]

if aristas_normales:

    nx.draw_networkx_edges(

        self.grafo, pos, edgelist=aristas_normales,

        arrows=True, arrowstyle="→", arrowsize=14,

        edge_color="#9AA5B1", width=1.4,

        connectionstyle="arc3,rad=0.18",

    )
```

```
# Etiquetas de distancia en cada arista
```

```
etiquetas_km = {
```

```
    (u, v): f"{formato_km(self.grafo.edges[u, v]['km'])} km"
```

```
    for u, v in self.grafo.edges
```

```
}
```

```
nx.draw_networkx_edge_labels(
```

```
    self.grafo, pos, edge_labels=etiquetas_km,
```

```
    font_size=7, font_color="#4A5568", )
```

```
# Nodos (ciudades)
```

```
    nodos_resaltados = set(ruta_resaltada) if ruta_resaltada else set()
```

```
    colores_nodos = [
```

```
        "#FF8C42" if n in nodos_resaltados else "#4C9FDB"
```

```
        for n in self.grafo.nodes
```

```
    ]
```

```
    nx.draw_networkx_nodes(
```

```
        self.grafo, pos, node_size=tamano_nodos,
```

```
        node_color=colores_nodos, edgecolors="#2C3E50",
```

```
        linewidths=1.2,    )
```

```
# Nombre de cada ciudad
```

```
nx.draw_networkx_labels(

    self.grafo, pos, font_size=9, font_weight="bold",

    font_color="#1A2B3C",

)

# Ruta resaltada (en rojo y más gruesa)

if ruta_resaltada:

    nx.draw_networkx_edges(

        self.grafo, pos, edgelist=list(aristas_resaltadas),

        arrows=True, arrowstyle="→", arrowsize=18,

        edge_color="#D64541", width=3.2,

        connectionstyle="arc3,rad=0.18",
```

```
)
```

```
print("Ruta resaltada en rojo: " + " → ".join(ruta_resaltada))
```

```
# Límites del mapa con margen
```

```
margen = 0.6
```

```
ax.set_xlim(min(todos_lons) - margen, max(todos_lons) + margen)
```

```
ax.set_ylim(min(todos_lats) - margen, max(todos_lats) + margen)
```

```
ax.set_aspect(1.09) # Compensa la distorsión de latitud en México
```

```
ax.grid(True, linestyle="--", linewidth=0.4, alpha=0.35, zorder=0)
```

```
ax.set_xlabel("Longitud (°)")
```

```
ax.set_ylabel("Latitud (°)")
```

```
ax.set_title("Red de Rutas Aéreas de México", fontsize=16, fontweight="bold")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 4. MENÚ INTERACTIVO
```

```
# -----
```

```
def mostrar_menu() → None:

    """Imprime el menú de opciones en la consola."""

    print("\n=====")

    print(" SISTEMA DE RUTAS AÉREAS DE MÉXICO")

    print(" Red nacional de vuelos (grafo dirigido)")

    print("=====")

    print("1. Agregar Ciudad")

    print("2. Agregar Ruta de Vuelo")

    print("3. Eliminar Ciudad")

    print("4. Eliminar Ruta de Vuelo")

    print("5. Mostrar Mapa de Rutas (Grafo)")
```



```
print("6. Ruta Más Corta entre Ciudades (Dijkstra)")
```

```
print("7. Distancia y Duración de Vuelo")
```

```
print("8. Listar Ciudades y Rutas")
```

```
print("9. Salir")
```

```
print("-----")
```

```
def seleccionar_ciudad_del_catalogo(red: RedVuelos) → str | None:
```

```
    """
```

```
    Permite elegir una ciudad del catálogo ampliado de México.
```

```
    Soporta búsqueda escribiendo parte del nombre (sin acentos).
```

Devuelve el nombre canónico o None si se cancela.

```
"""
```

```
print("\n--- Catálogo de Ciudades de México ---")
```

```
    busqueda = input("Escriba parte del nombre para buscar (ENTER = mostrar todas):  
").strip()
```

```
    if busqueda:
```

```
        clave = plegar_acentos(busqueda)
```

```
        resultados = [n for n in CATALOGO_CIUDADES if clave in plegar_acentos(n)]
```

```
        if not resultados:
```

```
            print(f"Sin coincidencias para '{busqueda}'. Verifique el nombre.")
```

```
    return None
```

```
else:
```

```
    resultados = sorted(CATALOGO_CIUDADES)
```

```
    for i, nombre in enumerate(resultados, 1):
```

```
        registrada = " (ya registrada)" if red.grafo.has_node(nombre) else ""
```

```
        print(f" {i:2d}. {nombre}{registrada}")
```

```
    eleccion = input("Seleccione el número de la ciudad (0 = cancelar): ").strip()
```

```
    try:
```

```
        indice = int(eleccion)
```

```
    except ValueError:
```

```
print("Error: Debe ingresar un número válido.")
```

```
return None
```

```
if indice == 0:
```

```
return None
```

```
if indice < 1 or indice > len(resultados):
```

```
print("Error: Número fuera de rango.")
```

```
return None
```

```
nombre = resultados[indice - 1]
```

```
if red.grafo.has_node(nombre):
```

```
print(f"Error: La ciudad '{nombre}' ya existe en el sistema.")
```

```
return None
```

```
return nombre
```

```
def principal() → None:
```

```
    """Bucle principal del programa con el menú interactivo."""
```

```
    red = RedVuelos()
```

```
    while True:
```

```
        mostrar_menu()
```

```
        opcion_str = input("Seleccione una opción (1-9): ").strip()
```

```
try:
```

```
    opcion = int(opcion_str)
```

```
    if opcion < 1 or opcion > 9:
```

```
        print("Error: Por favor, ingrese un número válido entre 1 y 9.")
```

```
        continue
```

```
except ValueError:
```

```
    print("Error: Entrada no válida. Debe ingresar un número.")
```

```
    continue
```

```
# 1. Agregar ciudad (desde el catálogo con búsqueda)
```

```
if opcion == 1:

    nombre = seleccionar_ciudad_del_catalogo(red)

    if nombre is not None:

        red.agregar_ciudad(nombre)


# 2. Agregar ruta de vuelo (con opción de ida y vuelta)


elif opcion == 2:

    origen = input("Ingrese la ciudad de origen: ")

    destino = input("Ingrese la ciudad de destino: ")

    red.agregar_ruta(origen, destino)

    redonda = input("¿Desea agregar también el vuelo de regreso? ")
```

```
"(s/n): ").strip().lower()
```

```
if redonda in ("s", "si", "sí"):
```

```
    red.agregar_ruta(destino, origen)
```

```
# 3. Eliminar ciudad
```

```
elif opcion == 3:
```

```
    ciudad = input("Ingrese la ciudad a eliminar: ")
```

```
    red.eliminar_ciudad(ciudad)
```

```
# 4. Eliminar ruta
```

```
elif opcion == 4:
```



```
origen = input("Ingrese la ciudad de origen de la ruta a eliminar: ")
```

```
destino = input("Ingrese la ciudad de destino de la ruta a eliminar: ")
```

```
red.eliminar_ruta(origen, destino)
```

```
# 5. Mostrar mapa del grafo
```

```
elif opcion == 5:
```

```
red.mostrar_grafo()
```

```
# 6. Ruta más corta (Dijkstra)
```

```
elif opcion == 6:
```

```
origen = input("Ingrese la ciudad de origen: ")
```

```
destino = input("Ingrese la ciudad de destino: ")

print("¿Qué desea minimizar?")

print(" 1. Distancia total (km)")

print(" 2. Número de escalas")

criterio = input("Seleccione (1-2): ").strip()

por_km = criterio != "2"

camino = red.ruta_mas_corta(origen, destino, por_km=por_km)

if camino is None:

    continue
```

```
km_total = sum(

    red.grafo.edges[u, v]["km"] for u, v in zip(camino, camino[1:])

)

escalas = len(camino) - 2

horas = (km_total / VELOCIDAD_CRUCERO_KMH

        + TIEMPO_TIERRA_HORAS * (len(camino) - 1))

print(f"\nRuta más corta encontrada:")

print(f"  Recorrido: {' → '.join(camino)}")

print(f"  Escalas: {escalas}")

print(f"  Distancia total: {formato_km(km_total)} km")
```

```
print(f" Duración estimada: {formato_duracion(horas)}")
```

```
dibujar = input("\n¿Desea ver la ruta resaltada en el mapa? "
```

```
"(s/n): ").strip().lower()
```

```
if dibujar in ("s", "si", "sí):
```

```
    red.mostrar_grafo(ruta_resaltada=camino)
```

```
# 7. Distancia y duración de vuelo
```

```
elif opcion == 7:
```

```
    origen = input("Ingrese la ciudad de origen: ")
```

```
    destino = input("Ingrese la ciudad de destino: ")
```

```
red.distancia_y_duracion(origen, destino)
```

```
# 8. Listar ciudades y rutas
```

```
elif opcion == 8:
```

```
red.listar_red()
```

```
# 9. Salir
```

```
elif opcion == 9:
```

```
print("Saliendo del sistema. ¡Hasta luego!")
```

```
break
```

```
if __name__ == "__main__":  
  
    try:  
  
        principal()  
  
    except KeyboardInterrupt:  
  
        print("\nSaliendo del sistema abruptamente...")  
  
    except Exception as e:  
  
        print(f"\nOcurrió un error inesperado: {e}")
```

5. Capturas de Pantalla

5.1 Captura 1: Menú del programa

Pasos: abrir una terminal, ubicarse en la carpeta del proyecto y ejecutar `python3 main.py`. La pantalla inicial muestra el menú completo con las 9 opciones.

[INSERTAR AQUÍ CAPTURA 1: Menú del programa]

5.2 Captura 2: Primer grafo con algunas ciudades

Pasos: en el menú, seleccionar la opción 5 (Mostrar Mapa de Rutas). Se muestra la red inicial con las 13 ciudades precargadas y sus 25 rutas sobre el mapa de México.

[INSERTAR AQUÍ CAPTURA 2: Primer grafo generado]

5.3 Captura 3: Modificación agregando ciudad y ruta

Pasos:

1. Opción 1 (Agregar Ciudad).
2. En la búsqueda escribir juar y presionar Enter; la lista filtrará a Ciudad Juárez.
3. Seleccionar el número correspondiente para agregarla.
4. Opción 2 (Agregar Ruta): origen Ciudad de México, destino Ciudad Juárez, y responder s a la pregunta de vuelo de regreso.
5. Opción 5 para ver el mapa actualizado.

[INSERTAR AQUÍ CAPTURA 3: Grafo modificado (agregando ciudad y ruta)]

5.4 Captura 4: Otra modificación del grafo (ruta más corta resaltada)

Pasos:

1. Opción 6 (Ruta Más Corta).
2. Origen Tijuana, destino Cancún, criterio 1 (distancia en km).
3. Responder s para ver la ruta resaltada en rojo sobre el mapa.

[INSERTAR AQUÍ CAPTURA 4: Grafo con la ruta más corta resaltada]

5.5 Captura 5: Otra modificación (eliminación de una ciudad)

Pasos:

1. Opción 3 (Eliminar Ciudad) e ingresar Oaxaca; el programa elimina la ciudad y todas sus rutas.
2. Opción 8 (Listar Ciudades y Rutas) para comprobar en consola que la ciudad ya no aparece.

3. Opción 5 para ver el mapa sin la ciudad eliminada.

[INSERTAR AQUÍ CAPTURA 5: Grafo después de eliminar una ciudad]

6. Conclusión

Qué aprendí realizando el proyecto. A lo largo del proyecto aprendí a modelar un problema real (las rutas de una aerolínea) con una estructura matemática abstracta: el grafo dirigido. Comprendí cómo se representa un grafo en código con la librería NetworkX, cómo almacenar información en los nodos y aristas (coordenadas y distancias), y cómo visualizarlo con Matplotlib. También aprendí a calcular distancias reales entre dos puntos con la fórmula de Haversine y a aplicar el algoritmo de Dijkstra para encontrar la ruta más corta, además de manejar entradas del usuario con validación de errores y organizar el código en funciones y clases.

Qué dificultades encontré. La principal dificultad fue la parte geográfica: obtener las coordenadas reales de las ciudades y el contorno del territorio mexicano, y lograr que cada ciudad apareciera dentro del mapa en su posición correcta. Otra dificultad fue representar visualmente las flechas de un grafo dirigido cuando existen vuelos en ambos sentidos entre dos ciudades, ya que se sobreponen; se resolvió curvando las flechas. También fue un reto validar todas las entradas del usuario (opciones inválidas, ciudades inexistentes, rutas duplicadas) para que el programa nunca se detuviera por un error.

Cómo se pueden aplicar los grafos en problemas reales. Los grafos tienen aplicación práctica casi ilimitada: las aerolíneas los usan para planear sus redes de vuelos y calcular itinerarios; los navegadores GPS para encontrar la ruta más rápida en carretera; las empresas de paquetería para optimizar sus rutas de reparto; las redes sociales para sugerir amigos; e incluso el internet se modela como un grafo gigante por el que viajan los datos. Este proyecto demuestra que con unas cuantas líneas de Python es posible construir una simulación de un sistema de transporte nacional, y que la teoría de grafos, lejos de ser un tema abstracto, resuelve problemas cotidianos.